Tier 4 — Expert: Coding Patterns

github.com/quanhua92/interview-prep

Interview Prep Playbook

July 2026

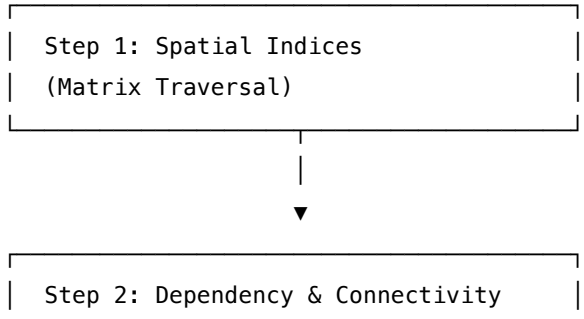
Contents

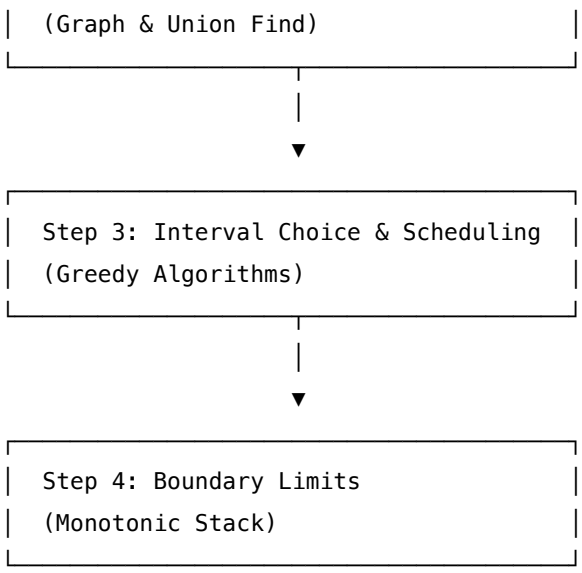
1 Tier 4: Expert — Coding Interview Preparation Roadmap

Welcome to the **Expert (Tier 4) Coding Roadmap**. This directory serves as your structured study guide and indexed reference repository for advanced/expert-level algorithmic design. These patterns cover complex multi-dimensional traversals, graph connectivity and dependency modeling, greedy heuristics, and monotonic limits.

1.1 Progression Roadmap

To build expert problem-solving intuition, start with spatial 2D matrix indexes, progress to network graphs and connectivity structures, then explore greedy choice intervals and monotonic boundary limits.





1.2 Detailed Learning Modules & File Index

Here is the complete catalog of the 24 Expert problems, categorized by pattern, and arranged in recommended learning order.

1.2.1 Module 1: Matrix Traversal

Goal: Manipulate 2D coordinate spaces, perform in-place spatial rotations, and resolve multi-source BFS grid maps.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Matrix Traversal	Rotate Image	LeetCode 48	Medium	Matrix transpose and horizontal reflection, in-place swaps
Matrix Traversal	Spiral Matrix	LeetCode 54	Medium	Index-boundary shrinking (top/bottom/left/right)
Matrix Traversal	Diagonal Traverse	LeetCode 498	Medium	Index sum grouping ($r + c = \text{const}$), direction flips
Matrix Traversal	01 Matrix	LeetCode 542	Medium	Multi-source BFS, 2D dynamic programming distance

1.2.2 Module 2: Dependency (Graph) & Connectivity (Union Find)

Goal: Model dependency relationships, sort topological constraints, check cycles, and evaluate partition groups.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Graph	Course Schedule	LeetCode 207	Medium	Cycle detection in directed graphs, Kahn's BFS, DFS colors
Graph	Course Schedule II	LeetCode 210	Medium	Topological sort order, in-degree arrays, DFS stack
Graph	Find the Town Judge	LeetCode 997	Easy	Vertex in-degree/out-degree balance, array index mapping
Union Find	Number of Connected Components	LeetCode 323	Medium	Disjoint Set Union (DSU) parent pointers, component count
Union Find	Redundant Connection	LeetCode 684	Medium	DSU cycle detection, path compression
Union Find	Satisfiability of Equality Equations	LeetCode 990	Medium	DSU partition equivalence, inequality validation check

1.2.3 Module 3: Greedy Choice & Scheduling

Goal: Make locally optimal choices (sorting, heaps, activity selections) to resolve globally optimal schedules and allocations.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Greedy	Jump Game	LeetCode 55	Medium	Maximum reachable index tracking, linear scan
Greedy	Gas Station	LeetCode 134	Medium	Continuous index sum, starting point tracking, total balance

Pattern	Problem Name	Source	Difficulty	Core Concepts
Greedy	Candy	LeetCode 135	Hard	Dual-pass rating differentials (left-to-right, right-to-left)
Greedy	Min Arrows to Burst Balloons	LeetCode 452	Medium	Interval sorting by end point, coordinate overlap greedy choice
Greedy	Assign Cookies	LeetCode 455	Easy	Dual sorting, two-pointer greedy allocation match
Greedy	IPO	LeetCode 502	Hard	Capital min-heap sort, project profit max-heap stream
Greedy	Task Scheduler	LeetCode 621	Medium	Max-frequency mapping math, idle slot counting
Greedy	Task Scheduler Multi-Machine	Custom	Hard	Multi-processor task sorting, earliest-available machine heaps

1.2.4 Module 4: Monotonic Stack (Boundary Limits)

Goal: Manage non-increasing/non-decreasing value/index arrays to find the nearest smaller/larger elements in $\mathcal{O}(n)$ time.

Pattern	Problem Name	Source	Difficulty	Core Concepts
Monotonic Stack	Largest Rectangle in Histogram	LeetCode 84	Hard	Non-decreasing height indices stack, width calculation
Monotonic Stack	132 Pattern	LeetCode 456	Medium	Decreasing stack value, third element candidates search
Monotonic Stack	Next Greater Element II	LeetCode 503	Medium	Circular array indexing (modulo $2n$), value stack

Pattern	Problem Name	Source	Difficulty	Core Concepts
Monotonic Stack	Daily Temperatures	LeetCode 739	Medium	Decreasing value indices stack, index offset calculations
Monotonic Stack	Car Fleet	LeetCode 853	Medium	Sorting by position, monotonic decrease of target times
Monotonic Stack	Sum of Subarray Minimums	LeetCode 907	Medium	Ple/Nle (previous/next lesser element) bounds, count products

---

1.3 Expert Tier Interview Strategy

1.3.1 1. Topological Sorting: Kahn's BFS vs. DFS

- **Kahn's BFS**: Preferred when you need to detect cycles explicitly and process items in exactly the order of their dependency satisfaction level (e.g. printing topological sequence). Utilizes an in-degree array.
- **DFS Color States**: Use color states (0=unvisited, 1=visiting, 2=visited) to identify back-edges (cycles) in directed graphs. DFS uses less memory in sparse trees but can overflow recursion limits.

1.3.2 2. Disjoint Set Union (DSU) Optimizations

- **Path Compression**: In the `find()` helper, recursively point child nodes directly to the root:

```
int find(int i) {
    if (parent[i] == i) return i;
    return parent[i] = find(parent[i]); // Path compression
}
```

- **Union by Rank/Size**: Merge the smaller tree under the larger tree to keep tree heights bounded by $\mathcal{O}(\alpha(n))$ (Inverse Ackermann complexity, practically ≤ 4).

1.3.3 3. Monotonic Stack index mappings

- **Store Indices, Not Values**: Always store the *indices* of the array elements on the monotonic stack rather than the raw values. Storing indices allows you to compute distances/widths (as in Daily Temperatures or Largest Rectangle) and fetch raw values directly (`nums[stack.top()]`).

- **Sentinel Elements**: To avoid checking for empty stacks during cleanup loops, append a sentinel element (e.g., `-1` or `0` for values) at the end of the input array. This forces the stack to flush and process all remaining items automatically.

2 Rotate Image

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 48, Glassdoor
- **Flashcards**: Matrix Traversal deck

2.1 Question Description

You are given an $n \times n$ 2D `matrix` representing an image, rotate the image by **90 degrees (clockwise)**.

You have to rotate the image **in-place**, which means you have to modify the input 2D matrix directly. **DO NOT** allocate another 2D matrix and do the rotation.

2.1.1 Examples

- **Input**: `matrix = [[1,2,3],[4,5,6],[7,8,9]]`
  - **Output**: `[[7,4,1],[8,5,2],[9,6,3]]`
- **Input**: `matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]`
  - **Output**: `[[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]`

2.2 Detailed Solution (C++)

There are two primary ways to perform an in-place 90-degree clockwise rotation of an $n \times n$ matrix:

2.2.1 Method 1: Transpose and Reverse (Elegant & Highly Readable)

1. **Transpose the matrix**: Swap elements across the main diagonal (`matrix[i][j]` with `matrix[j][i]` for $j > i$).
2. **Reverse each row**: Reverse the elements in each row horizontally.

Mathematically, transposing changes cell (i, j) to (j, i). Reversing the rows flips the columns, converting (j, i) to $(j, n-1-i)$. This maps exactly to a 90-degree clockwise rotation: $(i, j) \to (j, n-1-i)$.

2.2.2 Method 2: Shell/Ring-by-Ring Rotation (Single Pass)

Iterate layer by layer from the outermost boundary to the center. For each layer: * Perform a 4-way swap of corresponding elements: * `top-left` → `top-right` * `top-right` → `bottom-right` * `bottom-right` → `bottom-left` * `bottom-left` → `top-left`

This method avoids visiting cells multiple times and is more cache-friendly.

2.2.3 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>

class Solution {
public:
    void rotate(std::vector<std::vector<int>>& matrix) {
        // Edge Case: 1x1 or empty matrix
        if (matrix.empty() || matrix[0].empty()) {
            return;
        }

        int n = static_cast<int>(matrix.size());

        // Step 1: Transpose the matrix
        for (int i = 0; i < n; ++i) {
            for (int j = i + 1; j < n; ++j) {
                std::swap(matrix[i][j], matrix[j][i]);
            }
        }

        // Step 2: Reverse each row
        for (int i = 0; i < n; ++i) {
            std::reverse(matrix[i].begin(), matrix[i].end());
        }
    }

    // Alternative: Single-pass ring rotation
    void rotateSinglePass(std::vector<std::vector<int>>& matrix) {
        if (matrix.empty() || matrix[0].empty()) return;
        int n = static_cast<int>(matrix.size());

        for (int i = 0; i < n / 2; ++i) {
            int first = i;
            int last = n - 1 - i;
            for (int j = first; j < last; ++j) {
                int offset = j - first;

                int top = matrix[first][j]; // Save top

                // left -> top
                matrix[first][j] = matrix[last - offset][first];
```

```
                // bottom -> left
                matrix[last - offset][first] = matrix[last][last - offset];

                // right -> bottom
                matrix[last][last - offset] = matrix[j][last];

                // top -> right
                matrix[j][last] = top;
            }
        }
    }
};
```

2.3 Detailed Solution (Python)

2.3.1 Standard Python Production Code

```python
from typing import List

class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        """
        Rotates the matrix 90 degrees clockwise in-place.
        Do not return anything, modify matrix in-place instead.

        Time Complexity: O(N^2)
        Space Complexity: O(1)
        """
        if not matrix or not matrix[0]:
            return

        n = len(matrix)

        # Step 1: Transpose (swap across diagonal)
        for i in range(n):
            for j in range(i + 1, n):
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

        # Step 2: Reverse each row
        for row in matrix:
            row.reverse()
```

2.4 Python-Specific Complexities & Implementation Considerations

2.4.1 1. In-Place Modification vs Reference Assignment

- A common Python pitfall is reassigning the matrix variable: `matrix = [...]`.
- In LeetCode or standard APIs, the caller retains the reference to the original list object. Reassigning the local variable `matrix` only binds it to a new object within the local scope, leaving the caller's matrix unchanged.
- To modify the list content while keeping the same object, use slice assignment `matrix[:] = ...`, or modify elements in-place using loops / built-in methods like `row.reverse()`.

2.4.2 2. Pythonic Short-Hand: `matrix[:] = zip(*matrix[::-1])`

- In Python, you can write a one-liner to rotate the matrix:

```
matrix[:] = [list(row) for row in zip(*matrix[::-1])]
```

- **How it works**: `matrix[::-1]` reverses the rows vertically. `zip(*...)` unpacks the rows and zips them together, transposing the vertically reversed rows.
- **Production Note**: While elegant, this creates new tuple/list objects under the hood, violating the $\mathcal{O}(1)$ auxiliary space constraint. Explain this trade-off to the interviewer.

2.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n^2)$	We visit every cell in the $n \times n$ matrix a constant number of times (at most twice).
Space Complexity	$\mathcal{O}(1)$	All swaps are performed in-place using constant auxiliary memory.

2.6 Common Follow-Up Questions & How to Answer

2.6.1 Q1: How would you rotate the matrix 90 degrees counter-clockwise?

- **Answer**:
  - **Method 1**: Reverse each row horizontally first, then transpose.
  - **Method 2**: Transpose first, then reverse the rows vertically (reverse the matrix order of rows).
  - **Python Code (One-liner)**:
    ```
    matrix[:] = [list(row) for row in zip(*matrix)][::-1]
    ```

2.6.2 Q2: How would you rotate a non-square $m \times n$ matrix in-place?

- **Answer**: In-place rotation of a non-square matrix is highly challenging because the dimensions change (e.g., 3×4 becomes 4×3), shifting the memory layout of rows.

- If extra space is allowed, it is trivial: `new_matrix[j][m - 1 - i] = matrix[i][j]`.
- To do it in-place, it requires cycle-following permutations on a flattened array representation, which is a mathematically intensive process (similar to in-place transposition of rectangular matrices).

2.7 Pro-Tip: How to Impress the Interviewer

- **Highlight Cache Locality and Strides**:
  - Explain that during transposition, accessing `matrix[j][i]` involves column-wise traversal. In C++ and Python (where 2D vectors are arrays of pointers/lists), this causes cache misses because the next element is not contiguous in memory.
  - For large matrices, we can optimize cache locality using **blocked transposition** (processing the matrix in small sub-blocks of size $B \times B$, e.g., 16×16), which fits entirely within the L1/L2 cache lines.
- **Avoid Temporary List Creation**: Point out that solutions using `zip(*matrix)` are not truly in-place as they allocate $\mathcal{O}(n^2)$ memory for the tuples. The explicit double-loop swap method is preferred for production-grade, memory-constrained environments.

3 Spiral Matrix

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / AI Systems Architect
- **Source**: LeetCode 54, Glassdoor
- **Flashcards**: Matrix Traversal deck

3.1 Question Description

Given an $m \times n$ `matrix`, return all elements of the `matrix` in **spiral order**.

3.1.1 Examples

- **Input**: `matrix = [[1,2,3],[4,5,6],[7,8,9]]`
  - **Output**: `[1,2,3,6,9,8,7,4,5]`
- **Input**: `matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]`
  - **Output**: `[1,2,3,4,8,12,11,10,9,5,6,7]`

3.2 Detailed Solution (C++)

To traverse a matrix spirally, we can simulate the trajectory directly using **four boundaries/pointers**: 1. `top`: Initialized to `0` (index of the top row). 2. `bottom`: Initialized to `m - 1` (index of the bottom row). 3. `left`: Initialized to `0` (index of the leftmost column). 4. `right`: Initialized to `n - 1` (index of the rightmost column).

In each iteration of the outer loop: 1. Traverse from `left` to `right` along the `top` row. Then increment `top`. 2. Traverse from `top` to `bottom` along the `right` column. Then decrement `right`. 3. **Check Condition**: If `top <= bottom`, traverse from `right` to `left` along the `bottom` row. Then decrement `bottom`. 4. **Check Condition**: If `left <= right`, traverse from `bottom` to `top` along the `left` column. Then increment `left`.

We repeat this loop until the boundaries overlap (`top > bottom` or `left > right`). The conditional checks inside the loop are crucial to prevent double-processing elements when the remaining subgrid is a single row or a single column.

3.2.1 Standard C++ Production Code

```cpp
#include <vector>

class Solution {
public:
    std::vector<int> spiralOrder(std::vector<std::vector<int>>& matrix) {
        if (matrix.empty() || matrix[0].empty()) {
            return {};
        }

        int m = static_cast<int>(matrix.size());
        int n = static_cast<int>(matrix[0].size());

        std::vector<int> result;
        result.reserve(m * n); // Pre-allocate memory to avoid vector reallocations

        int top = 0;
        int bottom = m - 1;
        int left = 0;
        int right = n - 1;

        while (top <= bottom && left <= right) {
            // 1. Traverse Right (along the top boundary)
            for (int col = left; col <= right; ++col) {
                result.push_back(matrix[top][col]);
            }
            top++;

            // 2. Traverse Down (along the right boundary)
            for (int row = top; row <= bottom; ++row) {
                result.push_back(matrix[row][right]);
            }
            right--;

            // 3. Traverse Left (along the bottom boundary)
```

```
            if (top <= bottom) {
                for (int col = right; col >= left; --col) {
                    result.push_back(matrix[bottom][col]);
                }
                bottom--;
            }

            // 4. Traverse Up (along the left boundary)
            if (left <= right) {
                for (int row = bottom; row >= top; --row) {
                    result.push_back(matrix[row][left]);
                }
                left++;
            }
        }

        return result;
    }
};
```

3.3 Detailed Solution (Python)

3.3.1 Standard Python Production Code

```
from typing import List

class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        """
        Traverses a matrix in spiral order.

        Time Complexity: O(M * N)
        Space Complexity: O(1) auxiliary space (excluding the output array).
        """
        if not matrix or not matrix[0]:
            return []

        m, n = len(matrix), len(matrix[0])
        result: List[int] = []

        top, bottom = 0, m - 1
        left, right = 0, n - 1
```

```
    while top <= bottom and left <= right:
        # 1. Traverse Right
        for col in range(left, right + 1):
            result.append(matrix[top][col])
        top += 1

        # 2. Traverse Down
        for row in range(top, bottom + 1):
            result.append(matrix[row][right])
        right -= 1

        # 3. Traverse Left (Only if a valid row remains)
        if top <= bottom:
            for col in range(right, left - 1, -1):
                result.append(matrix[bottom][col])
            bottom -= 1

        # 4. Traverse Up (Only if a valid column remains)
        if left <= right:
            for row in range(bottom, top - 1, -1):
                result.append(matrix[row][left])
            left += 1

    return result
```

3.4 Python-Specific Complexities & Implementation Considerations

3.4.1 1. Pythonic One-Liner (Rotation Trick)

- Python allows an extremely concise solution using matrix transposition and vertical reversal:

```
def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
    return matrix and list(matrix.pop(0)) + self.spiralOrder(list(zip(*matrix))[::-1])
```

- **How it works**: It pops the first row, transposes the rest of the matrix, and reverses it vertically (which rotates the rest of the matrix 90 degrees counter-clockwise). This rotation brings the next spiral segment (the rightmost column) to the top row.
- **Production Note**: While clever, this approach is **unsuitable** for production. `matrix.pop(0)` is an $\mathcal{O}(m)$ operation because list elements must be shifted in memory. Additionally, zipping and slicing creates full copies of the subgrid in each step, taking $\mathcal{O}(m^2 n)$ time and $\mathcal{O}(m \cdot n)$ auxiliary space. Stick to boundary pointers for performance-critical systems.

3.4.2 2. Using `range` with Negative Step

- In Python, looping backwards is done using `range(start, stop, step)` where `step = -1`. Remember that the `stop` parameter is exclusive.
- To loop from `right` down to `left` inclusively, write `range(right, left - 1, -1)`.

3.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(m \times n)$	Every element of the matrix is visited exactly once.
Space Complexity	$\mathcal{O}(1)$	Auxiliary space is $\mathcal{O}(1)$ as we only maintain boundary pointers. The output list is not counted.

3.6 Common Follow-Up Questions & How to Answer

3.6.1 Q1: How would you solve Spiral Matrix II (LeetCode 59)?

- **Question**: Given a positive integer n, generate an $n \times n$ matrix filled with elements from 1 to n^2 in spiral order.
- **Answer**:
  1. Initialize an $n \times n$ grid with zeros.
  2. Maintain a counter starting at 1.
  3. Run the exact same boundary traversal simulation. Instead of reading from the matrix, set `matrix[r][c] = counter` and increment the counter.

3.6.2 Q2: What happens if the direction is counter-clockwise?

- **Answer**: Modify the sequence of transitions:
  1. Traverse Down along the left boundary (`top` to `bottom`).
  2. Traverse Right along the bottom boundary (`left` to `right`).
  3. Traverse Up along the right boundary (`bottom` to `top`).
  4. Traverse Left along the top boundary (`right` to `left`).
- Adjust boundary increments/decrements accordingly.

3.7 Pro-Tip: How to Impress the Interviewer

- **Pre-allocate Output Vectors**: In C++, always call `result.reserve(m * n)` before beginning the traversal. This prevents the vector from triggering multiple internal reallocations and memory copies as it grows.

- **Walk Through Single Row/Column Edge Cases**: Proactively trace your code with a 1×5 matrix or a 5×1 matrix. Explain how your `top <= bottom` and `left <= right` safety checks prevent the algorithm from traversing the same row or column in reverse, which is the most common bug candidates make.

4 Diagonal Traverse

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 498, Glassdoor
- **Flashcards**: Matrix Traversal deck

4.1 Question Description

Given an $m \times n$ matrix `mat`, return an array of all the elements of the array in a **diagonal order**.

4.1.1 Examples

- **Input**: `mat = [[1,2,3],[4,5,6],[7,8,9]]`
  - **Output**: `[1,2,4,7,5,3,6,8,9]`
- **Input**: `mat = [[1,2],[3,4]]`
  - **Output**: `[1,2,3,4]`

4.2 Detailed Solution (C++)

There are two primary approaches to traverse a matrix diagonally:

4.2.1 Method 1: Diagonal Grouping (Hash Map / Bucket Sort style)

Every element in cell (r, c) belongs to the diagonal $d = r + c$. * There are $m + n - 1$ diagonals. * We can group elements into buckets indexed by their diagonal index d. * For even-indexed diagonals, we reverse the elements before appending to the result. * *Downside*: Requires storing the entire matrix elements in intermediate lists, taking $\mathcal{O}(m \cdot n)$ auxiliary space.

4.2.2 Method 2: In-Place Boundary Simulation (Optimal $\mathcal{O}(1)$ Space)

We maintain a coordinate pointer `(r, c)` and a direction flag `up` (representing up-right movement). * If `up == true` (direction is up-right: `r--, c++`): * If we reach the rightmost column (`c == n - 1`), we must step down to the next row (`r++`) and flip the direction. * If we reach the topmost row (`r == 0`), we step right to the next column (`c++`) and flip the direction. * Otherwise, we move diagonally up-right: `r--, c++`. * If `up == false` (direction is down-left: `r++, c--`): * If we reach the bottommost row (`r == m - 1`), we step right to the next column (`c++`) and flip the direction. * If we reach the leftmost column (`c == 0`), we step down to the next row (`r++`) and flip the direction. * Otherwise, we move diagonally down-left: `r++, c--`.

This method runs in $\mathcal{O}(m \cdot n)$ time and requires strictly $\mathcal{O}(1)$ auxiliary space.

4.2.3 Standard C++ Production Code

```cpp
#include <vector>

class Solution {
public:
    std::vector<int> findDiagonalOrder(std::vector<std::vector<int>>& mat) {
        if (mat.empty() || mat[0].empty()) {
            return {};
        }

        int m = static_cast<int>(mat.size());
        int n = static_cast<int>(mat[0].size());

        std::vector<int> result;
        result.reserve(m * n);

        int r = 0, c = 0;
        bool up = true; // True for up-right direction, false for down-left

        for (int i = 0; i < m * n; ++i) {
            result.push_back(mat[r][c]);

            if (up) {
                // If moving up-right
                if (c == n - 1) {
                    // Reached right boundary -> move down and switch direction
                    r++;
                    up = false;
                } else if (r == 0) {
                    // Reached top boundary -> move right and switch direction
                    c++;
                    up = false;
                } else {
                    r--;
                    c++;
                }
            } else {
                // If moving down-left
                if (r == m - 1) {
                    // Reached bottom boundary -> move right and switch direction
                    c++;
```

```
                    up = true;
                } else if (c == 0) {
                    // Reached left boundary -> move down and switch direction
                    r++;
                    up = true;
                } else {
                    r++;
                    c--;
                }
            }
        }

        return result;
    }
};
```

4.3 Detailed Solution (Python)

4.3.1 Standard Python Production Code

```python
from typing import List

class Solution:
    def findDiagonalOrder(self, mat: List[List[int]]) -> List[int]:
        """
        Traverses a matrix diagonally in-place.

        Time Complexity: O(M * N)
        Space Complexity: O(1) auxiliary space (excluding the output array).
        """
        if not mat or not mat[0]:
            return []

        m, n = len(mat), len(mat[0])
        result: List[int] = []

        r, c = 0, 0
        up = True

        for _ in range(m * n):
            result.append(mat[r][c])

            if up:
```

```
            if c == n - 1:
                r += 1
                up = False
            elif r == 0:
                c += 1
                up = False
            else:
                r -= 1
                c += 1
        else:
            if r == m - 1:
                c += 1
                up = True
            elif c == 0:
                r += 1
                up = True
            else:
                r += 1
                c -= 1

    return result
```

4.4 Python-Specific Complexities & Implementation Considerations

4.4.1 1. Hash Map Dictionary grouping overhead

- A common Python approach uses a dictionary of lists: `diagonals = collections.defaultdict(list)`.
- While simple, this performs $\mathcal{O}(m \cdot n)$ lookups, inserts, and list-appends. In Python, dictionary hashing and dynamic list growth add significant runtime overhead and trigger high garbage collection frequency.
- The in-place pointer simulation method avoids any dictionary lookups and handles all updates via primitive pointer manipulations, making it significantly faster in Python.

4.4.2 2. Boundary Condition Ordering

- In the simulation, the order of boundary checks is critical.
- When moving up-right, we must check the right boundary (`c == n - 1`) **before** the top boundary (`r == 0`). If we hit the top-right corner of the matrix, both conditions are met. If we incorrectly process the top boundary first, we would increment `c` (getting `c == n`), resulting in an immediate out-of-bounds error on the next access.

4.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(m \times n)$	We visit every coordinate (r, c) in the matrix exactly once.
Space Complexity	$\mathcal{O}(1)$	The in-place pointer simulation operates with only a few scalar variables.

4.6 Common Follow-Up Questions & How to Answer

4.6.1 Q1: How would you generalize this to a 3D Tensor?

- **Answer**: In a 3D matrix (dimensions $X \times Y \times Z$), a diagonal slice corresponds to cells where the sum of indices $i + j + k = d$ is constant.
- We can group cells by their index sum $d \in [0, X+Y+Z-3]$. For each diagonal level d, we traverse the 2D plane slice, reversing the order on alternating steps to create a serpentine 3D diagonal traverse.

4.6.2 Q2: What if the matrix is too large to fit in memory?

- **Answer**: To process the matrix in a streaming fashion (e.g. from disk), we don't need the entire matrix in memory at once.
- Observe that a diagonal d only accesses cells where $r + c = d$, which corresponds to rows $r \in [\max(0, d-n+1), \min(m-1, d)]$.
- Thus, we only need to keep at most $\min(m, n)$ rows in memory at any point. We can load and discard rows dynamically as the active diagonal window slides forward.

4.7 Pro-Tip: How to Impress the Interviewer

- **Highlight Corner Bug Avoidance**: Explicitly point out the top-right and bottom-left corner overlap scenarios to the interviewer. Showing that you consciously ordered the check `if c == n - 1` before `elif r == 0` demonstrates deep attention to detail and proactive defense against out-of-bounds errors.
- **Discuss CPU Pipeline Friendliness**: Explain that while Method 2 is optimal in space, Method 1 (grouping) has simpler branching structures inside the loops. Method 2 has multiple branch points which can cause CPU **branch mispredictions**. For small matrices, Method 1 might actually execute faster due to predictable branch instructions, although it uses more memory.

5 01 Matrix

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / AI Systems Architect
- **Source**: LeetCode 542, Glassdoor

- **Flashcards**: Matrix Traversal deck

5.1 Question Description

Given an $m \times n$ binary matrix `mat`, return the distance of the nearest `0` for each cell.

The distance between two cells sharing a common edge is `1`.

5.1.1 Examples

- **Input**: `mat = [[0,0,0],[0,1,0],[0,0,0]]`
  - **Output**: `[[0,0,0],[0,1,0],[0,0,0]]`
- **Input**: `mat = [[0,0,0],[0,1,0],[1,1,1]]`
  - **Output**: `[[0,0,0],[0,1,0],[1,2,1]]`

5.2 Detailed Solution (C++)

There are two optimal ways to solve this problem:

5.2.1 Method 1: Multi-Source Breadth-First Search (BFS)

Instead of running a standard BFS from each `1` (which would take $\mathcal{O}((m \cdot n)^2)$ time), we run a single BFS starting from **all** `0`s simultaneously. 1. Initialize a `dist` matrix of the same size with -1 (unvisited). 2. Enqueue all coordinates of cells containing `0` and set their distance to `0`. 3. Pop coordinates from the queue. For each popped cell, check its 4-directional neighbors. 4. If a neighbor is unvisited (distance is -1), update its distance to `current_distance + 1` and push it to the queue. 5. This traverses the matrix in concentric levels starting from the `0`s, guaranteeing the shortest path is found in linear time.

5.2.2 Method 2: Two-Pass Dynamic Programming (Optimal $\mathcal{O}(1)$ Space)

A cell's shortest distance to a `0` is determined by its neighbors:

$$\text{dist}[r][c] = 1 + \min(\text{dist}[r-1][c], \text{dist}[r+1][c], \text{dist}[r][c-1], \text{dist}[r][c+1])$$

Since we cannot evaluate all four neighbors in a single traversal, we split the process into two passes: 1. **First Pass (Top-Left to Bottom-Right)**: Compute the distance using only top and left neighbors.

$$\text{dist}[r][c] = \min(\text{dist}[r][c], \min(\text{dist}[r-1][c], \text{dist}[r][c-1]) + 1)$$

2. **Second Pass (Bottom-Right to Top-Left)**: Compute the distance using only bottom and right neighbors.

$$\text{dist}[r][c] = \min(\text{dist}[r][c], \min(\text{dist}[r+1][c], \text{dist}[r][c+1]) + 1)$$

This method eliminates the queue allocation and runs in-place, achieving $\mathcal{O}(1)$ auxiliary space.

5.2.3 Standard C++ Production Code

```cpp
#include <vector>
#include <queue>
#include <algorithm>

class Solution {
public:
    // Method 1: Multi-source BFS
    std::vector<std::vector<int>> updateMatrixBFS(const std::vector<std::vector<int>>& mat) {
        if (mat.empty() || mat[0].empty()) return {};

        int m = static_cast<int>(mat.size());
        int n = static_cast<int>(mat[0].size());
        std::vector<std::vector<int>> dist(m, std::vector<int>(n, -1));
        std::queue<std::pair<int, int>> q;

        // Enqueue all 0 sources
        for (int r = 0; r < m; ++r) {
            for (int c = 0; c < n; ++c) {
                if (mat[r][c] == 0) {
                    dist[r][c] = 0;
                    q.push({r, c});
                }
            }
        }

        const int dr[] = {-1, 1, 0, 0};
        const int dc[] = {0, 0, -1, 1};

        // Standard BFS
        while (!q.empty()) {
            auto [r, c] = q.front();
            q.pop();

            for (int d = 0; d < 4; ++d) {
                int nr = r + dr[d];
                int nc = c + dc[d];

                // If neighbor is within bounds and unvisited
                if (nr >= 0 && nr < m && nc >= 0 && nc < n && dist[nr][nc] == -1) {
                    dist[nr][nc] = dist[r][c] + 1;
                    q.push({nr, nc});
                }
```

```
            }
        }

        return dist;
    }

    // Method 2: Two-pass DP (Optimal Space)
    std::vector<std::vector<int>> updateMatrix(std::vector<std::vector<int>>& mat) {
        if (mat.empty() || mat[0].empty()) return {};

        int m = static_cast<int>(mat.size());
        int n = static_cast<int>(mat[0].size());
        int maxDist = m + n; // Maximum possible distance

        // Pass 1: Top-Left to Bottom-Right
        for (int r = 0; r < m; ++r) {
            for (int c = 0; c < n; ++c) {
                if (mat[r][c] == 0) {
                    continue;
                }
                int top = (r > 0) ? mat[r - 1][c] : maxDist;
                int left = (c > 0) ? mat[r][c - 1] : maxDist;
                mat[r][c] = std::min(top, left) + 1;
            }
        }

        // Pass 2: Bottom-Right to Top-Left
        for (int r = m - 1; r >= 0; --r) {
            for (int c = n - 1; c >= 0; --c) {
                if (mat[r][c] == 0) {
                    continue;
                }
                int bottom = (r < m - 1) ? mat[r + 1][c] : maxDist;
                int right = (c < n - 1) ? mat[r][c + 1] : maxDist;
                mat[r][c] = std::min(mat[r][c], std::min(bottom, right) + 1);
            }
        }

        return mat;
    }
};
```

5.3 Detailed Solution (Python)

5.3.1 Standard Python Production Code

```python
from typing import List
from collections import deque

class Solution:
    def updateMatrix(self, mat: List[List[int]]) -> List[List[int]]:
        """
        Calculates the nearest distance to a 0 for each cell using Two-Pass DP.

        Time Complexity: O(M * N)
        Space Complexity: O(1) auxiliary space (modifying mat in-place).
        """
        if not mat or not mat[0]:
            return []

        m, n = len(mat), len(mat[0])
        max_dist = m + n

        # Pass 1: Top-Left to Bottom-Right
        for r in range(m):
            for c in range(n):
                if mat[r][c] == 0:
                    continue
                top = mat[r - 1][c] if r > 0 else max_dist
                left = mat[r][c - 1] if c > 0 else max_dist
                mat[r][c] = min(top, left) + 1

        # Pass 2: Bottom-Right to Top-Left
        for r in range(m - 1, -1, -1):
            for c in range(n - 1, -1, -1):
                if mat[r][c] == 0:
                    continue
                bottom = mat[r + 1][c] if r < m - 1 else max_dist
                right = mat[r][c + 1] if c < n - 1 else max_dist
                mat[r][c] = min(mat[r][c], min(bottom, right) + 1)

        return mat
```

5.4 Python-Specific Complexities & Implementation Considerations

5.4.1 1. Integer Limits and Initialization values

- In languages like C++, initializing values to `INT_MAX` can cause integer overflow when adding `+ 1` during the comparison.
- In Python, arbitrary precision integers won't overflow, but using `float('inf')` is common. However, `float('inf')` converts integers to floats, causing slow operations.
- Initializing with a safe upper bound (such as `m + n`, which is greater than any possible path distance in a grid) is preferred because it keeps all variables as primitive integers, maximizing execution speed.

5.4.2 2. Double-Ended Queue performance in BFS

- If you choose the BFS approach, avoid using a standard Python `list` as a queue. Inserting or removing from the front of a list takes $\mathcal{O}(V)$ time. Use `collections.deque` for true $\mathcal{O}(1)$ queue operations.

5.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(m \times n)$	Every cell is processed exactly twice in the DP approach (or visited at most once in BFS).
Space Complexity	$\mathcal{O}(1)$	The DP approach modifies the input matrix in-place. The BFS approach requires $\mathcal{O}(m \times n)$ space for the queue.

5.6 Common Follow-Up Questions & How to Answer

5.6.1 Q1: What is the relationship between Multi-Source BFS and Dijkstra's Algorithm?

- **Answer**: Multi-Source BFS is a special case of Dijkstra's Algorithm:
  - Dijkstra's algorithm operates on weighted graphs and uses a **Priority Queue** (Min-Heap) to extract the minimum distance node, taking $\mathcal{O}(E \log V)$ time.
  - Since the edge weights in this grid are all uniformly `1`, a standard FIFO queue naturally maintains its elements in monotonically increasing order of distance. Thus, we can replace the Min-Heap with a simple queue, reducing the time complexity to $\mathcal{O}(V + E) = \mathcal{O}(m \cdot n)$.

5.6.2 Q2: What if we can only move diagonally?

- **Answer**: If only diagonal steps are allowed:
  - The Manhattan distance changes to Chebyshev distance.
  - The grid splits into two independent, disjoint bipartite subgrids (like black and white squares on a chessboard).

- We change the neighbor direction offsets to `(-1, -1)`, `(-1, 1)`, `(1, -1)`, and `(1, 1)`.
- The DP passes would still work, but top/left would check `(r-1, c-1)` and `(r-1, c+1)` / `(r+1, c-1)`.

5.7 Pro-Tip: How to Impress the Interviewer

- **Proactively Propose the DP Method for Space Constraints**: Most candidates immediately jump to BFS. Explaining the DP approach shows you understand state dependency and trade-offs. Point out that the DP approach has much better **spatial locality of reference** because it processes rows sequentially, making it extremely friendly to CPU cache prefetchers.
- **Mention the Cache Performance of Grid Iterations**: Explain that in both passes of the DP approach, the outer loop iterates over `r` and the inner loop over `c`. This represents row-major ordering, which accesses adjacent cells sequentially in memory, avoiding cache misses. Never iterate columns in the outer loop (`c` then `r`) as it degrades performance due to stride-based cache misses.

6 Course Schedule

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / AI Systems Architect
- **Source**: LeetCode 207, Glassdoor
- **Flashcards**: Graph deck

6.1 Question Description

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`.

- For example, the pair `[1, 0]` indicates that to take course `1` you must first take course `0`.

Return `true` if you can finish all courses. Otherwise, return `false`.

6.1.1 Examples

- **Input**: `numCourses = 2`, `prerequisites = [[1,0]]`
  - **Output**: `true`
  - **Explanation**: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So it is possible.
- **Input**: `numCourses = 2`, `prerequisites = [[1,0],[0,1]]`
  - **Output**: `false`
  - **Explanation**: There are a total of 2 courses to take. To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.

6.2 Detailed Solution (C++)

The problem of finding if a course schedule is possible is equivalent to detecting a cycle in a directed graph. If the graph contains a cycle (e.g., $A \to B \to A$), then no topological ordering exists, and it is impossible to finish all courses.

We use **Kahn's Algorithm** (BFS-based Topological Sort): 1. **Build the Graph**: Represent the graph using an adjacency list and keep track of the in-degrees of all nodes. 2. **Initialize Queue**: Push all nodes with an in-degree of `0` (courses with no prerequisites) into a queue. 3. **Process Graph**: Pop a node from the queue, increment our visited count, and decrement the in-degree of all its neighbors. If a neighbor's in-degree drops to `0`, push it to the queue. 4. **Cycle Check**: If the visited count equals `numCourses`, then a valid topological order exists (return `true`). Otherwise, a cycle exists (return `false`).

6.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <queue>

class Solution {
public:
    bool canFinish(int numCourses, std::vector<std::vector<int>>& prerequisites) {
        // Build graph and calculate in-degrees
        std::vector<std::vector<int>> adj(numCourses);
        std::vector<int> inDegree(numCourses, 0);

        for (const auto& edge : prerequisites) {
            int course = edge[0];
            int prereq = edge[1];
            adj[prereq].push_back(course);
            inDegree[course]++;
        }

        // Initialize queue with courses having no prerequisites
        std::queue<int> q;
        for (int i = 0; i < numCourses; ++i) {
            if (inDegree[i] == 0) {
                q.push(i);
            }
        }

        int processedCount = 0;

        // Process courses topological-order BFS
```

```
        while (!q.empty()) {
            int current = q.front();
            q.pop();
            processedCount++;

            for (int neighbor : adj[current]) {
                if (--inDegree[neighbor] == 0) {
                    q.push(neighbor);
                }
            }
        }

        // If processedCount matches numCourses, we succeeded in processing all nodes without cycles
        return processedCount == numCourses;
    }
};
```

6.3 Detailed Solution (Python)

6.3.1 Standard Python Production Code

```python
from collections import deque
from typing import List

class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        """
        Determines if all courses can be finished using Kahn's algorithm (BFS topological sort).

        Time Complexity: O(V + E) where V is numCourses and E is prerequisites length.
        Space Complexity: O(V + E) for adjacency list and queue.
        """
        # Build graph and in-degree table
        graph = {i: [] for i in range(numCourses)}
        in_degree = [0] * numCourses

        for course, prereq in prerequisites:
            graph[prereq].append(course)
            in_degree[course] += 1

        # Push courses with 0 in-degrees to the queue
        queue = deque(i for i in range(numCourses) if in_degree[i] == 0)
```

```
        processed_count = 0

        while queue:
            node = queue.popleft()
            processed_count += 1

            for neighbor in graph[node]:
                in_degree[neighbor] -= 1
                if in_degree[neighbor] == 0:
                    queue.append(neighbor)

        return processed_count == numCourses
```

6.4 Python-Specific Complexities & Implementation Considerations

6.4.1 1. Choice of Queue Structure (`collections.deque` vs `list`)

- Using a standard Python list as a queue (`list.pop(0)`) is an $\mathcal{O}(n)$ operation because it requires shifting all subsequent elements in memory.
- Always use `collections.deque` (double-ended queue), which provides $\mathcal{O}(1)$ time complexity for both `append()` and `popleft()`.

6.4.2 2. Initializing Adjacency Lists: `dict` vs `list` of `lists`

- Since course IDs are continuous integers from `0` to `numCourses - 1`, we can use either a list of lists `[[] for _ in range(numCourses)]` or a dictionary.
- A list of lists typically has less memory overhead and faster lookups because array access by index in Python does not involve hash computations.

6.4.3 3. Avoiding Recursion Limits with DFS

- If cycle detection is implemented using Depth-First Search (DFS), recursion can reach a depth equal to the number of vertices V.
- Python's default recursion limit is 1000. For $numCourses = 2000$, a deep recursive path could raise a `RecursionError`. BFS (Kahn's algorithm) avoids recursion entirely and is safer for production.

6.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(V + E)$	Every vertex V and edge E (prerequisite) is processed exactly once.

Metric	Complexity	Description
Space Complexity	$\mathcal{O}(V + E)$	Adjacency list stores E edges, and the in-degree array / queue store at most V elements.

6.6 Common Follow-Up Questions & How to Answer

6.6.1 Q1: How would you detect a cycle using DFS instead of BFS?

- **Answer**: DFS-based cycle detection uses three states for each node:
  1. **White (0)**: Unvisited.
  2. **Gray (1)**: Visiting (currently in the recursion stack).
  3. **Black (2)**: Visited (all descendants processed).
- During DFS, if we transition to a neighbor that is in the **Gray** state, we have found a back-edge, indicating a cycle.
- **C++ DFS Implementation Snippet**:

```cpp
bool hasCycle(int node, const std::vector<std::vector<int>>& adj, std::vector<int>& state) {
    state[node] = 1; // Mark Gray
    for (int neighbor : adj[node]) {
        if (state[neighbor] == 1) return true; // Found cycle
        if (state[neighbor] == 0 && hasCycle(neighbor, adj, state)) return true;
    }
    state[node] = 2; // Mark Black
    return false;
}
```

6.6.2 Q2: What if we need to find the lexicographically smallest course schedule?

- **Answer**: Instead of a standard queue (which gives an arbitrary valid order depending on insertion), we can use a **Min-Priority Queue** (Min-Heap) to store courses with 0 in-degree.
- In C++, use `std::priority_queue<int, std::vector<int>, std::greater<int>>`. In Python, use the `heapq` module.
- Each time we pop from the heap, we process the smallest index course available, which guarantees a lexicographically sorted order.

6.7 Pro-Tip: How to Impress the Interviewer

- **Mention Vector-Based Queue Optimization**: Explain that in C++, instead of using `std::queue<int>`, we can use a `std::vector<int> q` and an index `front` acting as our read pointer.

Because Kahn's algorithm visits each node exactly once, the vector will grow to a maximum size of `numCourses`, saving allocation overhead and providing contiguous cache lines for the queue storage.

- **Memory Footprint & Locality**: Discuss how adjacency lists can be stored as a single contiguous array (`std::vector<int> edges` and `std::vector<int> offsets`) instead of `std::vector<std::vector<int>>` to reduce heap allocation fragmentation and improve CPU cache locality (L1/L2 cache prefetching).

7 Course Schedule II

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / AI Systems Architect
- **Source**: LeetCode 210, Glassdoor
- **Flashcards**: Graph deck

7.1 Question Description

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you must take course `bi` first if you want to take course `ai`.

- For example, the pair `[1, 0]` indicates that to take course `1` you must first take course `0`.

Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

7.1.1 Examples

- **Input**: `numCourses = 2`, `prerequisites = [[1,0]]`
  - **Output**: `[0,1]`
  - **Explanation**: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is `[0,1]`.
- **Input**: `numCourses = 4`, `prerequisites = [[1,0],[2,0],[3,1],[3,2]]`
  - **Output**: `[0,2,1,3]` (or `[0,1,2,3]`)
  - **Explanation**: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0. So one correct course order is `[0,1,2,3]`. Another correct ordering is `[0,2,1,3]`.
- **Input**: `numCourses = 1`, `prerequisites = []`
  - **Output**: `[0]`

7.2 Detailed Solution (C++)

This problem asks us to find a valid **topological sort** of the courses, represented as a Directed Acyclic Graph (DAG). If a cycle exists, topological sort is impossible, and we return an empty array.

We use **Kahn's Algorithm** (BFS-based Topological Sort): 1. **Construct Graph**: Form the adjacency list representing directed dependencies: `prereq -> course`. Calculate the in-degree of each course. 2. **Initialize Queue**: Push all courses with an in-degree of `0` into a `std::vector` that acts as both our BFS queue and our output array. 3. **Process BFS**: Iterate through the vector using a cursor index (`front`). For each popped course, decrement the in-degree of its dependencies. If a dependency's in-degree reaches `0`, append it to the vector. 4. **Verification**: If the size of the vector is equal to `numCourses`, return the vector. Otherwise, a cycle exists; return an empty vector `{}`.

7.2.1 Standard C++ Production Code

```cpp
#include <vector>

class Solution {
public:
    std::vector<int> findOrder(int numCourses, std::vector<std::vector<int>>& prerequisites) {
        std::vector<std::vector<int>> adj(numCourses);
        std::vector<int> inDegree(numCourses, 0);

        // Build the dependency graph
        for (const auto& edge : prerequisites) {
            int course = edge[0];
            int prereq = edge[1];
            adj[prereq].push_back(course);
            inDegree[course]++;
        }

        // We use a vector directly as our queue to avoid extra copies and memory allocations.
        // It will store the resulting topological order.
        std::vector<int> order;
        order.reserve(numCourses);

        for (int i = 0; i < numCourses; ++i) {
            if (inDegree[i] == 0) {
                order.push_back(i);
            }
        }

        // BFS traversal using a read cursor 'front'
        size_t front = 0;
        while (front < order.size()) {
            int current = order[front++];
            for (int neighbor : adj[current]) {
                if (--inDegree[neighbor] == 0) {
                    order.push_back(neighbor);
```

```
                }
            }
        }

        // If we processed all nodes, order is valid. Otherwise, there is a cycle.
        if (order.size() == static_cast<size_t>(numCourses)) {
            return order;
        }
        return {};
    }
};
```

7.3 Detailed Solution (Python)

7.3.1 Standard Python Production Code

```python
from collections import deque
from typing import List

class Solution:
    def findOrder(self, numCourses: int, prerequisites: List[List[int]]) -> List[int]:
        """
        Finds a valid course ordering using Kahn's algorithm (BFS-based Topological Sort).

        Time Complexity: O(V + E)
        Space Complexity: O(V + E)
        """
        # Build dependency graph
        graph = {i: [] for i in range(numCourses)}
        in_degree = [0] * numCourses

        for course, prereq in prerequisites:
            graph[prereq].append(course)
            in_degree[course] += 1

        # Initialize queue with in-degree 0 elements
        queue = deque(i for i in range(numCourses) if in_degree[i] == 0)
        order: List[int] = []

        while queue:
            node = queue.popleft()
            order.append(node)
```

```
            for neighbor in graph[node]:
                in_degree[neighbor] -= 1
                if in_degree[neighbor] == 0:
                    queue.append(neighbor)

        # Return order if all courses are visited, else empty list (cycle detected)
        return order if len(order) == numCourses else []
```

7.4 Python-Specific Complexities & Implementation Considerations

7.4.1 1. Vector/List Pre-allocation

- In Python, lists are dynamic arrays that dynamically resize as we append elements. Resizing has amortized $\mathcal{O}(1)$ time but can cause spikes in latency.
- Unlike C++'s `std::vector::reserve`, Python lists do not have an explicit reservation API. However, we can initialize a list of fixed size `order = [0] * numCourses` and maintain a write pointer, or simply use `append()` since Python's dynamic array resizing growth factor (roughly 1.125x) is optimized for general use.

7.4.2 2. Hash Map vs Array for In-Degrees

- Using a python `dict` to store in-degrees is possible but introduces hashing overhead. Because nodes are labelled from `0` to `numCourses - 1`, we should always use a contiguous list `[0] * numCourses` for constant-time lookup and update.

7.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(V + E)$	V represents the number of courses, and E is the number of prerequisite edges. Every vertex and edge is traversed once.
Space Complexity	$\mathcal{O}(V + E)$	The adjacency list stores E edges. The queue/order array and in-degree table require $\mathcal{O}(V)$ memory.

7.6 Common Follow-Up Questions & How to Answer

7.6.1 Q1: How would you find all possible topological sorts of the graph?

- **Answer**: To find all valid schedules, we use backtracking combined with Kahn's algorithm:

1. Keep track of unvisited nodes and the current state of in-degrees.
2. At each step, find all nodes with in-degree `0`.
3. For each such node, add it to the current path, decrement neighbor in-degrees, and recurse.
4. Backtrack by removing the node from the path and restoring neighbor in-degrees.

- This generates all valid topological sorts (which is $\mathcal{O}(V!)$ in the worst case for a completely disconnected graph).

7.6.2 Q2: How can we implement the cycle detection and topological sort using DFS?

- **Answer**: We can use DFS cycle detection with a post-order traversal:
  1. Maintain a `visited` state array (0: unvisited, 1: visiting, 2: visited).
  2. If we encounter a node in state 1, return `false` (cycle detected).
  3. Perform DFS on all unvisited nodes. When a node's DFS is complete, push it to a stack (or prepend to a list).
  4. Since we push a node to the stack only after visiting all its dependencies, the reverse of this stack gives the correct topological order.

7.7 Pro-Tip: How to Impress the Interviewer

- **Avoid Extra Queues in C++**: Point out that the output vector itself can serve as the queue in C++ Kahn's algorithm. By using a cursor index `front`, we eliminate the need for `std::queue` allocation. This guarantees $\mathcal{O}(1)$ auxiliary space if the output vector is not counted as auxiliary memory.
- **Explain Memory Fragmentation**: Discuss how nested vectors in C++ (`std::vector<std::vector<int>>`) cause cache misses because each inner vector is allocated separately on the heap. Suggest using a flattened array or a custom allocator if maximum cache performance is required in low-latency systems.

8 Find the Town Judge

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer / QA & Test Engineer
- **Source**: LeetCode 997, Glassdoor
- **Flashcards**: Graph deck

8.1 Question Description

In a town, there are `n` people labeled from `1` to `n`. There is a rumor that one of these people is secretly the town judge.

If the town judge exists, then: 1. The town judge trusts nobody. 2. Everybody (except for the town judge) trusts the town judge. 3. There is exactly one person that satisfies properties 1 and 2.

You are given an array `trust` where `trust[i] = [ai, bi]` representing that the person labeled `ai` trusts the person labeled `bi`.

Return the label of the town judge if the town judge exists and can be identified, or return `-1` otherwise.

8.1.1 Examples

- **Input**: `n = 2, trust = [[1,2]]`
  - **Output**: `2`
- **Input**: `n = 3, trust = [[1,3],[2,3]]`
  - **Output**: `3`
- **Input**: `n = 3, trust = [[1,3],[2,3],[3,1]]`
  - **Output**: `-1`

8.2 Detailed Solution (C++)

The town judge problem can be modeled as finding a node in a directed graph with an **in-degree** of $n-1$ and an **out-degree** of 0. * If person A trusts person B, there is a directed edge from $A \to B$. * This edge increases the out-degree of A and increases the in-degree of B.

Rather than tracking in-degree and out-degree separately, we can compute a single **trust score** for each person:

$$\text{trustScore}[i] = \text{inDegree}[i] - \text{outDegree}[i]$$

- If person i trusts someone, we decrement their score: `scores[a]--`.
- If person i is trusted by someone, we increment their score: `scores[b]++`.
- The town judge must have a score of exactly $n-1$ because they are trusted by all other $n-1$ people (in-degree $= n-1$) and trust no one (out-degree $= 0$).

8.2.1 Standard C++ Production Code

```cpp
#include <vector>

class Solution {
public:
    int findJudge(int n, std::vector<std::vector<int>>& trust) {
        // trust scores vector size n + 1 (1-indexed matching people labels)
        std::vector<int> scores(n + 1, 0);

        for (const auto& relation : trust) {
            int a = relation[0];
            int b = relation[1];
            scores[a]--; // Out-degree contribution
            scores[b]++; // In-degree contribution
        }
```

```
        for (int i = 1; i <= n; ++i) {
            if (scores[i] == n - 1) {
                return i;
            }
        }

        return -1;
    }
};
```

8.3 Detailed Solution (Python)

8.3.1 Standard Python Production Code

```python
from typing import List

class Solution:
    def findJudge(self, n: int, trust: List[List[int]]) -> int:
        """
        Finds the town judge using a single trust score array.

        Time Complexity: O(T + N) where T is length of trust and N is number of people.
        Space Complexity: O(N) for the score table.
        """
        scores = [0] * (n + 1)

        for a, b in trust:
            scores[a] -= 1
            scores[b] += 1

        for i in range(1, n + 1):
            if scores[i] == n - 1:
                return i

        return -1
```

8.4 Python-Specific Complexities & Implementation Considerations

8.4.1 1. 1-Based Indexing vs 0-Based Indexing

- The problem uses 1-based indexing for people (1 to n). In Python, it is cleaner to allocate an array of size $n+1$ rather than performing index shifts (e.g. `i - 1`), which can introduce off-by-one errors.

- The minor space overhead of 1 extra element is negligible compared to code readability and safety benefits.

8.4.2 2. Hash Map vs List for Low Space Overhead

- If n is very large (e.g. 10^9) but the trust array is small, allocating an array of size $n + 1$ would consume too much memory.
- In such a case, using a `collections.defaultdict(int)` to track trust scores is highly space-efficient, reducing auxiliary space to $\mathcal{O}(U)$ where U is the number of unique participants in the trust list.

8.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(N + T)$	We iterate through the trust array of size T once, then perform a linear scan of size N to find the judge.
Space Complexity	$\mathcal{O}(N)$	We store a trust score array of size $N + 1$.

8.6 Common Follow-Up Questions & How to Answer

8.6.1 Q1: What is the relationship between this and the "Celebrity Problem"?

- **Answer**: This problem is a variant of the **Celebrity Problem**. A celebrity is someone who is known by everyone but knows no one.
- While we are given a static trust array here, in the Celebrity Problem we are given a helper function `knows(A, B)`.
- We can find the celebrity in $\mathcal{O}(N)$ time and $\mathcal{O}(1)$ space using a **Two-Pointer elimination** strategy:
  1. Initialize candidate `c = 1`.
  2. For `i` from `2` to `n`: if `knows(c, i)`, then `c` knows someone and cannot be the celebrity. So update candidate `c = i`. If `knows(c, i)` is false, `i` is not known by `c`, so `i` cannot be the celebrity (celebrity is known by everyone). Thus, keep `c`.
  3. Verify the final candidate `c` by checking if `knows(c, i)` is false and `knows(i, c)` is true for all $i \neq c$.

8.6.2 Q2: What if we have a streaming flow of trust events?

- **Answer**: If trust relationships are added dynamically:
  1. Maintain the in-degree and out-degree using a hash map or direct-address table.
  2. Keep a pointer to the current candidate judge.
  3. If the candidate judge trusts someone (out-degree > 0), they are disqualified.

4. If a new edge is added, update degrees. Check if the updated destination node meets the conditions (out-degree == 0 and in-degree == $N-1$).

8.7 Pro-Tip: How to Impress the Interviewer

- **Avoid Double Allocations**: Many candidates allocate two separate arrays for `inDegree` and `outDegree`. Proactively explain that a single array tracking the difference `inDegree - outDegree` is sufficient because the only valid configuration for a judge is to have out-degree 0 and in-degree $n-1$, which unique maps to a score of $n-1$.
- **Mention Cache Line Locality**: Using a flat `std::vector` of size $n+1$ in C++ ensures all elements are contiguous in memory. This allows the CPU to fetch multiple scores into a single L1 cache line, making the second lookup pass extremely fast.

9 Number of Connected Components in an Undirected Graph

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 323, Glassdoor
- **Flashcards**: Union Find deck

9.1 Question Description

You have a graph of `n` nodes. You are given an integer `n` and an array `edges` where `edges[i] = [ai, bi]` indicates that there is an undirected edge between `ai` and `bi` in the graph.

Return the number of connected components in the graph.

9.1.1 Examples

- **Input**: `n = 5`, `edges = [[0,1],[1,2],[3,4]]`
  - **Output**: `2`
- **Input**: `n = 5`, `edges = [[0,1],[1,2],[2,3],[3,4]]`
  - **Output**: `1`

9.2 Detailed Solution (C++)

While this problem can be solved using standard graph traversals (DFS or BFS), the **Disjoint Set Union (DSU / Union-Find)** data structure is particularly well-suited. DSU operates directly on edges without needing to construct an explicit adjacency list, saving heap allocation overhead.

DSU Optimization Techniques: 1. **Path Compression**: In `find(x)`, we set the parent of `x` directly to the root of its set: `parent[x] = find(parent[x])`. This flattens the tree structure, ensuring subsequent

queries run in near constant time. 2. **Union by Rank (or Size)**: We attach the shorter tree (smaller rank) under the root of the taller tree. This keeps the tree height minimal.

We start with `n` individual components. For each edge, we attempt to `union` the two endpoints. If they belong to different components, we merge them and decrement the component count by 1. If they are already in the same component, we ignore the edge.

9.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <numeric>
#include <utility>

class UnionFind {
private:
    std::vector<int> parent;
    std::vector<int> rank;

public:
    UnionFind(int n) : parent(n), rank(n, 0) {
        std::iota(parent.begin(), parent.end(), 0); // parent[i] = i
    }

    int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]); // Path compression
        }
        return parent[x];
    }

    bool unite(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);
        if (rootX == rootY) {
            return false; // Already in the same component
        }

        // Union by rank
        if (rank[rootX] < rank[rootY]) {
            parent[rootX] = rootY;
        } else {
            parent[rootY] = rootX;
            if (rank[rootX] == rank[rootY]) {
                rank[rootX]++;
            }
```

```
        }
        return true;
    }
};

class Solution {
public:
    int countComponents(int n, std::vector<std::vector<int>>& edges) {
        UnionFind uf(n);
        int components = n;

        for (const auto& edge : edges) {
            if (uf.unite(edge[0], edge[1])) {
                components--;
            }
        }

        return components;
    }
};
```

9.3 Detailed Solution (Python)

9.3.1 Standard Python Production Code

```
from typing import List

class UnionFind:
    def __init__(self, n: int):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x: int) -> int:
        if self.parent[x] != x:
            # Recursive path compression
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]

    def union(self, x: int, y: int) -> bool:
        root_x = self.find(x)
        root_y = self.find(y)
        if root_x == root_y:
            return False
```

```
        # Union by rank
        if self.rank[root_x] < self.rank[root_y]:
            self.parent[root_x] = root_y
        else:
            self.parent[root_y] = root_x
            if self.rank[root_x] == self.rank[root_y]:
                self.rank[root_x] += 1
        return True

class Solution:
    def countComponents(self, n: int, edges: List[List[int]]) -> int:
        """
        Calculates the number of connected components in an undirected graph using Union-Find.

        Time Complexity: O(V + E * alpha(V)) where alpha is the Inverse Ackermann function.
        Space Complexity: O(V) to store parent and rank.
        """
        uf = UnionFind(n)
        components = n

        for u, v in edges:
            if uf.union(u, v):
                components -= 1

        return components
```

9.4 Python-Specific Complexities & Implementation Considerations

9.4.1 1. Recursion Limit during Path Compression

- In Python, deep recursion can hit the system stack limit. However, because of Union by Rank, the height of any Union-Find tree is bounded by $\mathcal{O}(\log n)$, and with path compression, it is effectively $\mathcal{O}(\alpha(n)) \leq 5$ in practice.
- Therefore, the recursive implementation of `find` is safe and will not cause a `RecursionError` for any practical graph size.

9.4.2 2. Space Efficiency over DFS/BFS

- DFS/BFS requires constructing an adjacency list representation of the graph (e.g. `dict[int, list[int]]`). In Python, this involves creating V list objects and $2E$ references, introducing significant memory overhead.
- The DSU implementation processes edges in-place and only requires two flat lists (`parent` and `rank`) of size V, resulting in a highly optimized memory footprint.

9.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(V + E \cdot \alpha(V))$	Union and find operations run in amortized $\mathcal{O}(\alpha(V))$ time, where α is the Inverse Ackermann function (practically constant, ≤ 5).
Space Complexity	$\mathcal{O}(V)$	Requires $\mathcal{O}(V)$ space to maintain the parent and rank arrays. No adjacency list or recursion stack is needed.

9.6 Common Follow-Up Questions & How to Answer

9.6.1 Q1: What is the relative trade-off between DFS/BFS and Union-Find for this problem?

- **Answer**:
  - **DFS/BFS**: Best if we already have the graph represented as an adjacency list. The time complexity is strictly $\mathcal{O}(V + E)$. However, if we are given raw edges, constructing the adjacency list requires extra time and $\mathcal{O}(V + E)$ auxiliary memory.
  - **Union-Find**: Best if we receive the edges in a stream or if memory is restricted. It processes edges directly, requiring only $\mathcal{O}(V)$ auxiliary memory. It is also easily parallelizable and supports dynamic graph updates.

9.6.2 Q2: How can we track the size of the largest connected component during unions?

- **Answer**: We can replace the `rank` array with a `size` array initialized to `1` for all nodes.
  - When we perform `union(root_x, root_y)`, we update: `size[root_x] += size[root_y]` and set `parent[root_y] = root_x`.
  - We track a global `max_size` variable that is updated to `max(max_size, size[root_x])` whenever a union occurs.

9.7 Pro-Tip: How to Impress the Interviewer

- **Explain Inverse Ackermann** α: Don't just say the time complexity is $\mathcal{O}(1)$ or $\mathcal{O}(\log^* n)$ (log-star). Clarify that path compression combined with union by rank gives $\mathcal{O}(\alpha(n))$, where α is the Inverse Ackermann function. Mention that for all practical values (up to $2^{2^{65536}}$), $\alpha(n) \leq 5$, making it virtually constant.

- **Demonstrate Direct Heap Savings**: Highlight that because Union-Find does not build an adjacency list, it avoids allocating many small container nodes (vector fragments in C++ or list/dict buckets in Python). This improves L1 cache spatial locality and drastically reduces GC (garbage collection) pressure in managed environments like Python or JVM.

10 Redundant Connection

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 684, Glassdoor
- **Flashcards**: Union Find deck

10.1 Question Description

In this problem, a tree is an undirected graph that is connected and has no cycles.

You are given a graph that started as a tree with `n` nodes labeled from `1` to `n`, with one additional edge added. The added edge has two different vertices chosen from `1` to `n`, and was not an edge that already existed. The graph is represented as an array `edges` of length `n` where `edges[i] = [ai, bi]` indicates that there is an edge between nodes `ai` and `bi` in the graph.

Return an edge that can be removed so that the resulting graph is a tree of `n` nodes. If there are multiple answers, return the answer that occurs last in the input.

10.1.1 Examples

- **Input**: `edges = [[1,2],[1,3],[2,3]]`
  - **Output**: `[2,3]`
- **Input**: `edges = [[1,2],[2,3],[3,4],[1,4],[1,5]]`
  - **Output**: `[1,4]`

10.2 Detailed Solution (C++)

This problem asks us to find an edge that forms a cycle in an undirected graph. Since we want to return the *last* edge in the input that creates the cycle, we can process the edges one by one in the order they appear.

We use **Disjoint Set Union (DSU / Union-Find)**: 1. Initialize a DSU structure for n nodes, where each node is its own parent. 2. Iterate through each edge `[u, v]`. 3. Query `find(u)` and `find(v)`. * If `find(u) == find(v)`, it means u and v are already connected in the same component. Adding the edge `[u, v]` would create a cycle. Since we process edges in the given order, this must be the redundant connection. We immediately return `[u, v]`. * If they are in different components, we merge them using `unite(u, v)`.

10.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <numeric>
#include <utility>

class UnionFind {
private:
    std::vector<int> parent;
    std::vector<int> rank;

public:
    UnionFind(int size) : parent(size), rank(size, 0) {
        std::iota(parent.begin(), parent.end(), 0);
    }

    int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]); // Path compression
        }
        return parent[x];
    }

    bool unite(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);
        if (rootX == rootY) {
            return false; // Cycle detected (already connected)
        }

        // Union by rank
        if (rank[rootX] < rank[rootY]) {
            parent[rootX] = rootY;
        } else {
            parent[rootY] = rootX;
            if (rank[rootX] == rank[rootY]) {
                rank[rootX]++;
            }
        }
        return true;
    }
};

class Solution {
```

```cpp
public:
    std::vector<int> findRedundantConnection(std::vector<std::vector<int>>& edges) {
        int n = static_cast<int>(edges.size());
        // Nodes are labeled 1 to n, so size is n + 1
        UnionFind uf(n + 1);

        for (const auto& edge : edges) {
            if (!uf.unite(edge[0], edge[1])) {
                return edge; // This edge creates the cycle
            }
        }

        return {};
    }
};
```

10.3 Detailed Solution (Python)

10.3.1 Standard Python Production Code

```python
from typing import List

class UnionFind:
    def __init__(self, size: int):
        self.parent = list(range(size))
        self.rank = [0] * size

    def find(self, x: int) -> int:
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # Path compression
        return self.parent[x]

    def union(self, x: int, y: int) -> bool:
        root_x = self.find(x)
        root_y = self.find(y)
        if root_x == root_y:
            return False

        if self.rank[root_x] < self.rank[root_y]:
            self.parent[root_x] = root_y
        else:
            self.parent[root_y] = root_x
            if self.rank[root_x] == self.rank[root_y]:
```

```
                self.rank[root_x] += 1
        return True

class Solution:
    def findRedundantConnection(self, edges: List[List[int]]) -> List[int]:
        """
        Finds the redundant connection that completes a cycle.

        Time Complexity: O(N * alpha(N))
        Space Complexity: O(N)
        """
        n = len(edges)
        uf = UnionFind(n + 1)

        for u, v in edges:
            if not uf.union(u, v):
                return [u, v]

        return []
```

10.4 Python-Specific Complexities & Implementation Considerations

10.4.1 1. Pre-allocation of `parent` list size

- Since the node labels are 1-indexed and we are given n edges, the number of nodes is exactly n (because a tree of n nodes has $n-1$ edges, and adding one edge results in n nodes and n edges).
- Allocating the `parent` list to `n + 1` elements prevents off-by-one errors when indexing.

10.4.2 2. Functional Closure vs OOP UnionFind Class

- In Python, calling method on objects incurs minor overhead due to dynamic attribute lookup.
- In highly constrained performance environments, you can define `parent` and `find` as local variables/closures within the outer function. However, the class-based structure is preferred in production code for clean separation of concerns and testability.

10.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(N \cdot \alpha(N))$	We process N edges. Each Union-Find operation takes $\mathcal{O}(\alpha(N))$ time.

Metric	Complexity	Description
Space Complexity	$\mathcal{O}(N)$	The DSU structure uses `parent` and `rank` arrays of size $N + 1$.

10.6 Common Follow-Up Questions & How to Answer

10.6.1 Q1: How would you solve this for a directed graph? (Redundant Connection II)

- **Answer**: In a directed graph, the problem is more complex because a cycle is not the only way to violate tree properties; a node having two parents is also invalid.
  1. First scan and check if any node has two parents (in-degree == 2). Record the two incoming edges.
  2. If such a node exists, try removing the second edge and run Union-Find on the rest. If no cycle is detected, the second edge is the answer. If a cycle is detected, the first edge must be the answer.
  3. If no node has two parents, then the graph contains a simple directed cycle. Use standard Union-Find to find the edge that closes the cycle.

10.6.2 Q2: Can we solve Redundant Connection using DFS?

- **Answer**: Yes, we can process edges one by one. For each edge `[u, v]`, run DFS starting from `u` to see if we can reach `v`.
  - If `v` is reachable, then `[u, v]` is the redundant edge.
  - If not, add the edge `[u, v]` to our adjacency list and continue.
  - **Drawback**: DFS takes $\mathcal{O}(V+E)$ per edge, making the total complexity $\mathcal{O}(V(V+E)) = \mathcal{O}(V^2)$, which is slower than Union-Find's near-linear complexity.

10.7 Pro-Tip: How to Impress the Interviewer

- **Explain the Incremental Connectivity Advantage**: Emphasize that Union-Find is an **online/incremental** algorithm. If edges were streamed to us one by one over a network socket, Union-Find could detect the cycle instantly without needing to traverse the historical graph. DFS, on the other hand, would have to re-traverse the entire graph from scratch for every incoming edge.
- **Mention Space Locality Optimization**: In C++, explain that instead of two vectors (`parent` and `rank`), we can store both in a single `std::vector<int> parent` where the root nodes store their rank (or tree size) as a negative value (e.g. `parent[root] = -size`). This reduces allocations, cuts cache footprint by half, and improves processor cache efficiency.

11 Satisfiability of Equality Equations

- **Category**: Coding
- **Difficulty**: Medium

- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 990, Glassdoor
- **Flashcards**: Union Find deck

11.1 Question Description

You are given an array of strings `equations` that represent relationships between variables where each string `equations[i]` is of length `4` and takes one of two different forms: `"xi==yi"` or `"xi!=yi"`. Here, `xi` and `yi` are lowercase letters (not necessarily different) that represent one-letter variable names.

Return `true` if it is possible to assign integers to variable names so as to satisfy all the given equations, or `false` otherwise.

11.1.1 Examples

- **Input**: `equations = ["a==b","b!=a"]`
  - **Output**: `false`
  - **Explanation**: If we assign $a = 1$ and $b = 1$, the first equation is satisfied, but not the second. There is no way to assign the variables to satisfy both.
- **Input**: `equations = ["b==a","a==b"]`
  - **Output**: `true`
  - **Explanation**: We could assign $a = 1$ and $b = 1$ to satisfy both equations.

11.2 Detailed Solution (C++)

The problem asks us to determine if a system of equality and inequality constraints is consistent. 1. **Equality is an Equivalence Relation**: Equality is reflexive, symmetric, and transitive. This means we can group variables that are equal to each other into connected components (equivalence classes). 2. **Disjoint Set Union (DSU)**: * In the first pass, we process all equations of the form `xi == yi` and union the sets containing `xi` and `yi`. * In the second pass, we process all equations of the form `xi != yi`. If `find(xi) == find(yi)`, they are in the same equivalence class, which means we have determined they must be equal. Since the equation states they are *not* equal, this is a contradiction. We return `false`. * If no contradiction is found after checking all inequality equations, we return `true`.

Since all variables are lowercase letters (`'a'` to `'z'`), the total number of elements is fixed at 26. This makes DSU extremely fast and space-efficient.

11.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <string>
#include <numeric>
#include <utility>
```

```cpp
class UnionFind {
private:
    int parent[26];
    int rank[26];

public:
    UnionFind() {
        for (int i = 0; i < 26; ++i) {
            parent[i] = i;
            rank[i] = 0;
        }
    }

    int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]); // Path compression
        }
        return parent[x];
    }

    void unite(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);
        if (rootX != rootY) {
            // Union by rank
            if (rank[rootX] < rank[rootY]) {
                parent[rootX] = rootY;
            } else {
                parent[rootY] = rootX;
                if (rank[rootX] == rank[rootY]) {
                    rank[rootX]++;
                }
            }
        }
    }
};

class Solution {
public:
    bool equationsPossible(std::vector<std::string>& equations) {
        UnionFind uf;

        // First pass: Process all equality equations
        for (const auto& eq : equations) {
```

```
            if (eq[1] == '=') {
                int u = eq[0] - 'a';
                int v = eq[3] - 'a';
                uf.unite(u, v);
            }
        }

        // Second pass: Validate against inequality equations
        for (const auto& eq : equations) {
            if (eq[1] == '!') {
                int u = eq[0] - 'a';
                int v = eq[3] - 'a';
                if (uf.find(u) == uf.find(v)) {
                    return false; // Contradiction: u and v must be equal, but are declared unequal
                }
            }
        }

        return true;
    }
};
```

11.3 Detailed Solution (Python)

11.3.1 Standard Python Production Code

```
from typing import List

class UnionFind:
    def __init__(self):
        self.parent = list(range(26))
        self.rank = [0] * 26

    def find(self, x: int) -> int:
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # Path compression
        return self.parent[x]

    def union(self, x: int, y: int) -> None:
        root_x = self.find(x)
        root_y = self.find(y)
        if root_x != root_y:
            if self.rank[root_x] < self.rank[root_y]:
```

```
                self.parent[root_x] = root_y
            else:
                self.parent[root_y] = root_x
                if self.rank[root_x] == self.rank[root_y]:
                    self.rank[root_x] += 1

class Solution:
    def equationsPossible(self, equations: List[str]) -> bool:
        """
        Determines the satisfiability of equality and inequality constraints.

        Time Complexity: O(N * alpha(26)) = O(N) where N is the number of equations.
        Space Complexity: O(1) auxiliary space (fixed at 26 variables).
        """
        uf = UnionFind()

        # Pass 1: Union variables that are equal
        for eq in equations:
            if eq[1] == '=':
                u = ord(eq[0]) - ord('a')
                v = ord(eq[3]) - ord('a')
                uf.union(u, v)

        # Pass 2: Check for contradictions against unequal variables
        for eq in equations:
            if eq[1] == '!':
                u = ord(eq[0]) - ord('a')
                v = ord(eq[3]) - ord('a')
                if uf.find(u) == uf.find(v):
                    return False

        return True
```

11.4 Python-Specific Complexities & Implementation Considerations

11.4.1 1. Fixed Range vs Map for Dynamic Variables

- Because the variable set is constrained to lowercase letters `a-z`, we map variables directly to the integer range 0–25 using `ord(char) - ord('a')`.
- If variables were arbitrary alphanumeric strings or variable-length identifiers, we would use a Python `dict` to map strings to unique sequential IDs, or implement the parent/rank arrays directly using dictionaries:

  ```
  self.parent = {}
  ```

```
    def find(self, x: str) -> str:
        if x not in self.parent:
            self.parent[x] = x
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
        return self.parent[x]
```

11.4.2 2. String Slicing and String Indices

- In Python, strings are immutable arrays of Unicode code points. Accessing a character by index, e.g., `eq[0]` and `eq[3]`, is an $\mathcal{O}(1)$ operation.
- Avoid using string slicing (like `eq[0:1]`) as it creates a new string object, which is slower and uses more heap memory.

11.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(N)$	N is the number of equations. DSU operations on a set of size 26 run in $\mathcal{O}(\alpha(26))$ which is $O(1)$ constant time.
Space Complexity	$\mathcal{O}(1)$	The parent and rank arrays are fixed at size 26, using constant auxiliary space.

11.6 Common Follow-Up Questions & How to Answer

11.6.1 Q1: What if equations can be entered dynamically in a stream, and we need to validate query relationships on the fly?

- **Answer**:
  - For equalities, Union-Find handles dynamic streaming perfectly.
  - For inequalities, we can maintain an **adjacently list of inequalities** for each disjoint set representative.
  - When we receive `A != B`, we find their representatives `root_A` and `root_B`. If `root_A == root_B`, we reject. Otherwise, we record `root_A` and `root_B` as mutually unequal by adding each to the other's "unequal" set.
  - When we receive `A == B`, we find `root_A` and `root_B`. Before merging, we verify that `root_B` is not in the "unequal" set of `root_A`. If they are compatible, we union them and merge their "unequal" sets.

11.6.2 Q2: Can we solve this problem using DFS/BFS?

- **Answer**: Yes. We can treat equalities as undirected edges and build an adjacency list.
  1. Perform DFS/BFS on all unvisited nodes to assign a unique component ID (or color) to each variable.
  2. For each inequality `u != v`, verify if `u` and `v` have different component IDs. If they have the same ID, return `false`.
  - **DSU vs DFS**: DSU is much cleaner here as it does not require building an explicit adjacency list and has a smaller memory overhead.

11.7 Pro-Tip: How to Impress the Interviewer

- **Mention Compile-Time Array Sizes**: In C++, declare parent and rank arrays as fixed-size arrays on the stack (`int parent[26]`) rather than heap-allocated vectors (`std::vector<int>`). Stack allocation is instantaneous (essentially adjusting the stack pointer), avoids cache fragmentation, and does not require heap cleanups.
- **Explain the Equivalence Partitioning concept**: Describe how equality relations partitions the variable space into disjoint equivalence classes. This connects the problem directly to the mathematical foundation of equivalence relations, showing a solid academic and theoretical background.

12 Jump Game

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 55, Glassdoor
- **Flashcards**: Greedy deck

12.1 Question Description

You are given an integer array `nums`. You are initially positioned at the array's **first index**, and each element in the array represents your maximum jump length at that position.

Return `true` if you can reach the last index, or `false` otherwise.

12.1.1 Examples

- **Input**: `nums = [2,3,1,1,4]`
  - **Output**: `true`
  - **Explanation**: Jump 1 step from index 0 to 1, then 3 steps to the last index.
- **Input**: `nums = [3,2,1,0,4]`
  - **Output**: `false`
  - **Explanation**: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes it impossible to reach the last index.

12.2 Detailed Solution (C++)

The problem can be solved in a single pass using a **Greedy Algorithm**. We maintain a variable `max_reach` representing the farthest index we can reach at any point in time. 1. We iterate through the array from left to right. 2. At each index `i`, we first check if `i` is greater than `max_reach`. If it is, then index `i` is unreachable, meaning we cannot proceed further or reach the end. We immediately return `false`. 3. Otherwise, we update `max_reach` to be $\max(\text{maxReach}, i + \text{nums}[i])$. 4. If the loop completes successfully, it means we could reach every index along the way, including the last one. We return `true`.

12.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>

class Solution {
public:
    bool canJump(std::vector<int>& nums) {
        // Edge Case: An empty vector or single-element vector
        if (nums.empty()) {
            return false;
        }

        int max_reach = 0;
        const int n = static_cast<int>(nums.size());

        for (int i = 0; i < n; ++i) {
            // If the current index is beyond the farthest reachable index, we fail
            if (i > max_reach) {
                return false;
            }

            // Update the farthest reachable index
            max_reach = std::max(max_reach, i + nums[i]);

            // Early exit: if we can already reach or exceed the last index
            if (max_reach >= n - 1) {
                return true;
            }
        }

        return true;
    }
};
```

12.3 Detailed Solution (Python)

12.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using an optimized iterative loop.

```python
from typing import List

class Solution:
    def canJump(self, nums: List[int]) -> bool:
        """
        Determines if you can reach the last index in a list of maximum jump lengths.

        Time Complexity: O(N)
        Space Complexity: O(1)
        """
        if not nums:
            return False

        max_reach = 0
        n = len(nums)

        for i, jump in enumerate(nums):
            # If the current index is beyond the farthest reachable index, we fail
            if i > max_reach:
                return False

            # Update the farthest reachable index
            # Inline conditional is faster than calling built-in max() in Python
            current_reach = i + jump
            if current_reach > max_reach:
                max_reach = current_reach

            # Early exit: if we can already reach or exceed the last index
            if max_reach >= n - 1:
                return True

        return True
```

12.4 Python-Specific Complexities & Implementation Considerations

12.4.1 1. Built-in `max()` Function Overhead

- In Python, calling the built-in `max()` function within a loop introduces function-call overhead. For performance-critical code where $N \leq 10^5$, replacing `max_reach = max(max_reach, i + jump)` with an inline `if i + jump > max_reach:` check avoids the overhead of frame creation and variable lookup in Python's VM, leading to a measurable speedup.

12.4.2 2. Space Overhead of Recursion / Memoization

- While Jump Game can be formulated as a Dynamic Programming / DFS + Memoization problem ($O(N^2)$ or $O(N)$ with memoized states), doing so in Python can easily hit the recursion depth limit (default 1000) for large test cases.
- Furthermore, storing the memoization table (e.g., via `@lru_cache` or a `list`/`dict`) degrades auxiliary space complexity to $\mathcal{O}(N)$. The greedy approach runs in $\mathcal{O}(1)$ space and is highly preferred.

12.4.3 3. Enumeration vs Range Indexing

- Using `enumerate(nums)` is preferred in Python over `range(len(nums))` when both index and value are needed. `enumerate` is implemented in C and yields a tuple containing the index and object reference directly, minimizing index lookup overhead inside the loop.

12.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We perform a single linear scan of the array.
Space Complexity	$\mathcal{O}(1)$	Only a constant number of integer variables are used.

12.6 Common Follow-Up Questions & How to Answer

12.6.1 Q1: How do we return the minimum number of jumps to reach the last index? (Jump Game II / LeetCode 45)

- **Answer**: We can solve this greedily as well. Instead of just tracking the absolute farthest index, we track the boundary of the current jump level (`current_end`) and the farthest we can reach with the next jump (`farthest`). Every time we reach `current_end`, we increment our jump count and update `current_end = farthest`.
- **C++ Snippet**:

```cpp
int jump(const std::vector<int>& nums) {
    int jumps = 0, current_end = 0, farthest = 0;
```

```
        for (int i = 0; i < (int)nums.size() - 1; ++i) {
            farthest = std::max(farthest, i + nums[i]);
            if (i == current_end) {
                jumps++;
                current_end = farthest;
            }
        }
        return jumps;
    }
```

12.6.2 Q2: What if we want to print one actual sequence of indices leading to the last index?

- **Answer**: We can keep a parent array `parent` where `parent[i]` stores the index we jumped from to reach `i`. Since we only need any valid path, we can construct the path during our greedy traversal. Alternatively, we can traverse backward: start from the last index, find the smallest index `j` that can reach `i`, set `parent[i] = j`, and update `i = j` until `i == 0`.

12.7 Pro-Tip: How to Impress the Interviewer

- **Mention Backward Traversal**: Mention that the problem can also be solved by traversing the array from right to left, keeping track of the "closest index that can reach the destination" (initially `n-1`). If we find an index `i` such that `i + nums[i] >= goal`, we update `goal = i`. At the end, if `goal == 0`, return `true`. This backward greedy approach is equivalent but sometimes easier to reason about.
- **Early-Exit Optimization**: Always include the early exit condition `if (max_reach >= n - 1) return true;`. For arrays where you can jump to the end immediately (e.g., `[100000, 1, 2, ...]`), the algorithm stops on the first iteration rather than scanning the remaining elements.
- **Explain Hardware Prefetching**: In C++, scanning a contiguous `std::vector` leverages the CPU's L1/L2 cache prefetchers. The greedy forward scan accesses contiguous memory blocks, making it highly cache-friendly compared to recursion or pointer-based structures.

13 Gas Station

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 134, Glassdoor
- **Flashcards**: Greedy deck

13.1 Question Description

There are n gas stations along a circular route, where the amount of gas at the i-th station is `gas[i]`.

You have a car with an unlimited gas tank and it costs `cost[i]` of gas to travel from the i-th station to its next $(i+1)$-th station. You begin the journey with an empty tank at one of the gas stations.

Given two integer arrays `gas` and `cost`, return the starting gas station's index if you can travel around the circuit once in the clockwise direction, otherwise return `-1`. If there exists a solution, it is guaranteed to be unique.

13.1.1 Examples

- **Input**: `gas = [1,2,3,4,5]`, `cost = [3,4,5,1,2]`
  - **Output**: `3`
  - **Explanation**:
    - Start at station 3 (index 3) and fill up with 4 units of gas. Your tank = 0 + 4 = 4.
    - Travel to station 4. Your tank = 4 - 1 + 5 = 8.
    - Travel to station 0. Your tank = 8 - 2 + 1 = 7.
    - Travel to station 1. Your tank = 7 - 3 + 2 = 6.
    - Travel to station 2. Your tank = 6 - 4 + 3 = 5.
    - Travel to station 3. The cost is 5. Your gas is just enough to travel back to station 3.
    - Therefore, return 3 as the starting index.
- **Input**: `gas = [2,3,4]`, `cost = [3,4,3]`
  - **Output**: `-1`
  - **Explanation**:
    - You can't start at station 0 or 1, as there is not enough gas to travel to the next station.
    - Let's start at station 2 and fill up with 4 units of gas. Your tank = 0 + 4 = 4.
    - Travel to station 0. Your tank = 4 - 3 + 2 = 3.
    - Travel to station 1. Your tank = 3 - 3 + 3 = 3.
    - You cannot travel back to station 2, as it requires 4 units of gas but you only have 3.
    - Therefore, you can't travel around the circuit once no matter where you start.

13.2 Detailed Solution (C++)

The problem is solved using a **Greedy Algorithm** in a single pass.

13.2.1 Mathematical Proof of Correctness

1. **Total Surplus Condition**: If the sum of all elements in `gas` is greater than or equal to the sum of all elements in `cost` (i.e., $\sum(\text{gas}[i] - \text{cost}[i]) \geq 0$), a valid starting gas station is guaranteed to exist. If the total sum is negative, it is mathematically impossible to complete the circuit, and we immediately return `-1`.
2. **Greedy Start Reset**: Let's say we start at station A and reach station B where we can no longer proceed to $B+1$ because our `current_surplus` becomes negative. This implies that for any station C between A and B ($A \leq C < B$), starting at C will also fail to reach $B+1$.
   - *Proof*: When starting at A, our gas tank when we arrived at C was ≥ 0. If we couldn't make it to $B+1$ starting from A with a boost (or neutral tank) at C, then starting at C with 0 gas

will definitely fail. Thus, we can safely reset our candidate starting station to $B + 1$ and reset our `current_surplus` to 0.

13.2.2 Standard C++ Production Code

```cpp
#include <vector>
#include <numeric>

class Solution {
public:
    int canCompleteCircuit(std::vector<int>& gas, std::vector<int>& cost) {
        int total_surplus = 0;
        int current_surplus = 0;
        int start = 0;
        const int n = static_cast<int>(gas.size());

        for (int i = 0; i < n; ++i) {
            int diff = gas[i] - cost[i];
            total_surplus += diff;
            current_surplus += diff;

            // If the tank goes negative, we cannot start at 'start' or any station up to 'i'.
            if (current_surplus < 0) {
                start = i + 1;       // Next candidate starting station
                current_surplus = 0;  // Reset tank for the new candidate
            }
        }

        // If the total gas accumulated is less than total cost, no solution exists.
        return total_surplus >= 0 ? start : -1;
    }
};
```

13.3 Detailed Solution (Python)

13.3.1 Standard Python Production Code

Below is the optimized, type-hinted Python implementation using `zip` to avoid indexing overhead.

```python
from typing import List

class Solution:
    def canCompleteCircuit(self, gas: List[int], cost: List[int]) -> int:
        """
        Finds the unique starting gas station's index to complete the circular route.
```

```
    Time Complexity: O(N)
    Space Complexity: O(1)
    """
    total_surplus = 0
    current_surplus = 0
    start = 0

    # zip() allows us to loop through both lists concurrently and is faster in Python
    # than indexing gas[i] and cost[i] directly because it minimizes indexing overhead.
    for i, (g, c) in enumerate(zip(gas, cost)):
        diff = g - c
        total_surplus += diff
        current_surplus += diff

        if current_surplus < 0:
            start = i + 1
            current_surplus = 0

    return start if total_surplus >= 0 else -1
```

13.4 Python-Specific Complexities & Implementation Considerations

13.4.1 1. Loop Optimization via `zip()`

- In Python, accessing lists using index lookup (e.g., `gas[i]`) within a loop requires pointer indirection and bounds checking.
- Using `zip(gas, cost)` creates an iterator in C that yields pairs of values directly. Combined with `enumerate()`, it provides both the index and values at maximum performance.

13.4.2 2. Large Sum Integer Overflow Safety

- In languages like C or C++, accumulating `total_surplus` can theoretically overflow fixed-width integers if $n = 10^5$ and gas/cost values are extremely large (though not possible under LeetCode constraints where values $\leq 10^4$).
- In Python, integers have arbitrary precision, so accumulator overflow is never an issue. However, in C++, using `int` is safe here since the maximum possible total surplus is $10^5 \times 10^4 = 10^9$, which fits comfortably inside a standard 32-bit signed integer (limit $\approx 2.14 \times 10^9$).

13.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Single pass over the gas and cost arrays.
Space Complexity	$\mathcal{O}(1)$	Auxiliary space is constant since we only store a few scalar variables.

13.6 Common Follow-Up Questions & How to Answer

13.6.1 Q1: What if the solution is not unique? How do we find the starting index that maximizes the remaining gas at the end?

- **Answer**: The greedy logic works because if a solution exists, any starting point that completes the circuit must accumulate positive surplus throughout the journey. To find the starting index that maximizes the final remaining gas (or the minimum gas dip during the trip), we can look at the cumulative prefix sum of `gas[i] - cost[i]`. The starting index that minimizes the minimum cumulative valley of our prefix sum will leave us with the highest gas cushion. In fact, the station immediately *after* the global minimum of the prefix sum is always the optimal start.

13.6.2 Q2: Can we solve this problem using Kadane's algorithm?

- **Answer**: Yes, conceptually. The problem is equivalent to finding a contiguous subarray of size n in a doubled array (representing circularity) that has no negative prefix sums. Our greedy restart mechanism mirrors Kadane's reset step (`if current_surplus < 0: current_surplus = 0`).

13.7 Pro-Tip: How to Impress the Interviewer

- **Explain the Prefix Sum Minimum Interpretation**: Impress the interviewer by mentioning that this problem is equivalent to finding the minimum value in the cumulative prefix sums of `diff = gas[i] - cost[i]`. If we plot the prefix sum of `diff` as a curve, the starting station must be the index right after the global minimum. This is because starting there shifts the curve upward, ensuring that the cumulative sum never drops below zero.
- **Explicit Loop Bounds and Casts**: In C++, casting `gas.size()` to a signed `int` via `static_cast<int>` or using a `std::size_t` loop index prevents compiler warnings regarding signed/unsigned comparisons.
- **Single-Pass Guarantee**: Emphasize that because we check `total_surplus >= 0` at the very end, we do not need to perform a second pass to simulate the circular trip. This mathematical guarantee saves 50% of instruction cycles.

14 Candy

- **Category**: Coding

- **Difficulty**: Hard
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 135, Glassdoor
- **Flashcards**: Greedy deck

14.1 Question Description

There are n children standing in a line. Each child is assigned a rating value given in the integer array `ratings`.

You are giving candies to these children subjected to the following requirements: 1. Each child must have at least one candy. 2. Children with a higher rating get more candies than their neighbors.

Return the *minimum number of candies* you need to have to distribute the candies to the children.

14.1.1 Examples

- **Input**: `ratings = [1,0,2]`
  - **Output**: `5`
  - **Explanation**: You can allocate to the first, second, and third child with 2, 1, and 2 candies respectively.
- **Input**: `ratings = [1,2,2]`
  - **Output**: `4`
  - **Explanation**: You can allocate to the first, second, and third child with 1, 2, and 1 candies respectively. The third child gets 1 candy because it satisfies the above two conditions (it has rating 2, which is equal to its left neighbor's rating, not higher).

14.2 Detailed Solution (C++)

The problem can be solved efficiently using a **Two-Pass Greedy Algorithm**: 1. **Initialize**: Create a list/array `candies` of size n, filled with `1` (since every child must get at least one candy). 2. **Left-to-Right Pass**: Iterate from index 1 to $n-1$. If `ratings[i] > ratings[i-1]`, we must give child `i` more candies than child `i-1`. Thus, we update `candies[i] = candies[i-1] + 1`. This satisfies the condition for left neighbors. 3. **Right-to-Left Pass**: Iterate from index $n-2$ down to 0. If `ratings[i] > ratings[i+1]`, child `i` must have more candies than child `i+1`. We update `candies[i] = std::max(candies[i], candies[i+1] + 1)`. This satisfies the condition for right neighbors while preserving the correctness of the left neighbor conditions. 4. **Sum**: The sum of all elements in `candies` is the final answer.

14.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <numeric>
#include <algorithm>
```

```cpp
class Solution {
public:
    int candy(std::vector<int>& ratings) {
        const int n = static_cast<int>(ratings.size());
        if (n <= 1) {
            return n;
        }

        std::vector<int> candies(n, 1);

        // First pass: satisfy left neighbors
        for (int i = 1; i < n; ++i) {
            if (ratings[i] > ratings[i - 1]) {
                candies[i] = candies[i - 1] + 1;
            }
        }

        // Second pass: satisfy right neighbors
        // Iterate backwards and take the max of current and candies[i+1] + 1
        for (int i = n - 2; i >= 0; --i) {
            if (ratings[i] > ratings[i + 1]) {
                candies[i] = std::max(candies[i], candies[i + 1] + 1);
            }
        }

        // Return the sum of all candies
        return std::accumulate(candies.begin(), candies.end(), 0);
    }
};
```

14.3 Detailed Solution (Python)

14.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using the two-pass greedy strategy.

```python
from typing import List

class Solution:
    def candy(self, ratings: List[int]) -> int:
        """
        Calculates the minimum number of candies required to satisfy neighbor rating conditions.

        Time Complexity: O(N)
```

```
    Space Complexity: O(N)
    """
    n = len(ratings)
    if n <= 1:
        return n

    candies = [1] * n

    # Left-to-right pass
    for i in range(1, n):
        if ratings[i] > ratings[i - 1]:
            candies[i] = candies[i - 1] + 1

    # Right-to-left pass
    for i in range(n - 2, -1, -1):
        if ratings[i] > ratings[i + 1]:
            # Inline comparison is faster than max() function in Python
            r_candies = candies[i + 1] + 1
            if r_candies > candies[i]:
                candies[i] = r_candies

    return sum(candies)
```

14.4 Python-Specific Complexities & Implementation Considerations

14.4.1 1. Built-in `sum()` Utility

- Summing values in a Python list can be done in-loop or via `sum(candies)`. The built-in `sum()` function is implemented in optimized C and iterates over the list elements without PyObject type checking overhead at each step, making it much faster than an explicit `for x in candies: total += x` loop.

14.4.2 2. Avoiding `max()` Function Overhead

- As with the Jump Game problem, the Python VM incurs frame-allocation overhead when calling the built-in `max()` function. Replacing `candies[i] = max(candies[i], candies[i+1] + 1)` with a simple `if` condition speeds up execution, which is crucial when $N = 2 \times 10^4$.

14.4.3 3. Space Allocation Overhead

- Initializing `candies = [1] * n` is the standard way to allocate a list of size n in Python. This creates n references to the integer object `1`. If memory consumption is a bottleneck, a single-pass $\mathcal{O}(1)$ space slope-tracking method can be implemented instead.

14.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Two linear scans over the array of size n.
Space Complexity	$\mathcal{O}(n)$	We allocate a dynamic array of size n to track candy distributions.

14.6 Common Follow-Up Questions & How to Answer

14.6.1 Q1: Can we solve the Candy problem in $\mathcal{O}(1)$ auxiliary space?

- **Answer**: Yes. We can track slopes (peak and valley patterns) as we walk from left to right:
  - An increasing ratings sequence forms an **up-slope**.
  - A decreasing ratings sequence forms a **down-slope**.
  - When ratings are equal, the slope resets to 0.
  - We maintain the length of the up-slope and down-slope. When we descend, the height of the valley increases by 1 for each child in the down-slope. If the down-slope length exceeds the peak height, we must retroactively add candies to the peak.
- **Python Code**:

```python
def candy_constant_space(ratings: List[int]) -> int:
    n = len(ratings)
    if n <= 1:
        return n

    candies = 1
    up = 0
    down = 0
    peak = 0

    for i in range(1, n):
        if ratings[i] > ratings[i - 1]:
            up += 1
            down = 0
            peak = up
            candies += 1 + up
        elif ratings[i] == ratings[i - 1]:
            up = 0
            down = 0
            peak = 0
```

```
            candies += 1
        else:
            down += 1
            up = 0
            # If down-slope matches or exceeds the peak, the peak needs to be taller.
            candies += down + (1 if down > peak else 0)

    return candies
```

14.6.2 Q2: What if the children are arranged in a circular formation?

- **Answer**: In a circular arrangement, index 0 and index $n-1$ are neighbors. To solve this:
  1. First, find a child whose rating is less than or equal to both of their neighbors. Such a child is a local minimum (valley) and must receive exactly 1 candy.
  2. Unroll the circular array starting from this child.
  3. Run the standard two-pass greedy algorithm on the unrolled linear array.
  4. Reconnect the circular boundary checks to verify consistency at the endpoints.

14.7 Pro-Tip: How to Impress the Interviewer

- **Proactively Discuss the $O(1)$ Space Optimization**: Before diving straight into coding, explain both the two-pass $\mathcal{O}(n)$ space solution and the single-pass $\mathcal{O}(1)$ space slope-tracking solution. This demonstrates a deep mastery of array parsing algorithms.
- **Explain Cache Locality**: In C++, accessing `candies` in the backward pass (`i = n-2` down to `0`) works against the hardware prefetcher's forward expectations, but since the array fits entirely in the L1 cache (2×10^4 integers ≈ 80 KB), cache miss latency is negligible.
- **Use `std::accumulate`**: In C++, rather than writing a manual summing loop, use `<numeric>`'s `std::accumulate`. It is highly expressive, and modern compilers can vectorize it using SIMD (Single Instruction, Multiple Data) instructions automatically.

15 Minimum Number of Arrows to Burst Balloons

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer
- **Source**: LeetCode 452, Glassdoor
- **Flashcards**: Greedy deck

15.1 Question Description

There are some spherical balloons taped onto a flat wall that represents the XY-plane. The balloons are represented as a 2D integer array `points` where `points[i] = [xstart, xend]` denotes a balloon whose

horizontal diameter stretches between `xstart` and `xend`. You do not know the exact y-coordinates of the balloons.

Arrows can be shot up directly vertically (in the positive y-direction) from different points along the x-axis. A balloon with `xstart` and `xend` is burst by an arrow shot at x if $x_{start} \leq x \leq x_{end}$. There is no limit to the number of arrows that can be shot. A shot arrow keeps traveling up infinitely, bursting any balloons in its path.

Given the array `points`, return the *minimum number of arrows* that must be shot to burst all balloons.

15.1.1 Examples

- **Input**: `points = [[10,16],[2,8],[1,6],[7,12]]`
  - **Output**: `2`
  - **Explanation**: The balloons can be burst by 2 arrows:
    - Shoot an arrow at $x = 6$, bursting the balloons `[2,8]` and `[1,6]`.
    - Shoot an arrow at $x = 11$, bursting the balloons `[10,16]` and `[7,12]`.
- **Input**: `points = [[1,2],[3,4],[5,6],[7,8]]`
  - **Output**: `4`
  - **Explanation**: One arrow needs to be shot for each balloon for a total of 4 arrows.
- **Input**: `points = [[1,2],[2,3],[3,4],[4,5]]`
  - **Output**: `2`
  - **Explanation**: The balloons can be burst by 2 arrows:
    - Shoot an arrow at $x = 2$, bursting the balloons `[1,2]` and `[2,3]`.
    - Shoot an arrow at $x = 4$, bursting the balloons `[3,4]` and `[4,5]`.

15.2 Detailed Solution (C++)

This is a classic **Interval Scheduling / Greedy Algorithm** problem: 1. If we sort the intervals (balloons) by their **end coordinates**, we can always greedily place an arrow at the end coordinate of the first balloon. 2. Placing the arrow at the end coordinate E of the balloon maximizes the chance of hitting subsequent balloons because any balloon starting at or before E must overlap with this arrow's path. 3. For each subsequent balloon, if its start coordinate is greater than our current arrow's position, we need a new arrow. We update the arrow's position to the end coordinate of this new balloon. 4. Otherwise, the balloon is burst by the existing arrow, and we continue.

15.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>

class Solution {
public:
    int findMinArrowShots(std::vector<std::vector<int>>& points) {
        if (points.empty()) {
```

```
            return 0;
        }

        // Sort balloons by their end coordinate (points[i][1])
        std::sort(points.begin(), points.end(), [](const std::vector<int>& a, const std::vector<int>& b) noe
            return a[1] < b[1];
        });

        int arrows = 1;
        // Using coordinate value directly as the position of the arrow
        int current_end = points[0][1];

        const size_t n = points.size();
        for (size_t i = 1; i < n; ++i) {
            // If the start of the current balloon is greater than the current arrow position,
            // we must shoot a new arrow.
            if (points[i][0] > current_end) {
                arrows++;
                current_end = points[i][1];
            }
        }

        return arrows;
    }
};
```

15.3 Detailed Solution (Python)

15.3.1 Standard Python Production Code

Below is the type-hinted, memory-optimized Python implementation.

```
from typing import List

class Solution:
    def findMinArrowShots(self, points: List[List[int]]) -> int:
        """
        Calculates the minimum number of arrows to burst all balloons.

        Time Complexity: O(N log N)
        Space Complexity: O(1) auxiliary (sorting in Python takes O(N) space for Timsort)
        """
        if not points:
            return 0
```

```
        # Sort in-place by the end coordinate to avoid O(N) list copy
        points.sort(key=lambda x: x[1])

        arrows = 1
        current_end = points[0][1]

        # Use an iterator to avoid list slicing points[1:] which allocates O(N) memory
        points_iter = iter(points)
        next(points_iter)  # Skip the first element

        for start, end in points_iter:
            if start > current_end:
                arrows += 1
                current_end = end

        return arrows
```

15.4 Python-Specific Complexities & Implementation Considerations

15.4.1 1. Avoiding Slice Allocation Overhead

- Using `points[1:]` inside the loop creates a shallow copy of the list, which consumes $\mathcal{O}(N)$ additional memory.
- By converting the list to an iterator with `iter(points)` and skipping the first element using `next()`, we can iterate through the remaining elements with zero auxiliary memory allocations.

15.4.2 2. Timsort Space Complexity

- Python's built-in `sort()` (and `sorted()`) uses Timsort, which has a worst-case space complexity of $\mathcal{O}(N)$ because it keeps track of runs in temporary arrays. In C++, `std::sort` uses Introsort, which runs in $\mathcal{O}(\log N)$ auxiliary space.

15.4.3 3. Comparator Subtraction Overflow Gotcha

- In some languages, developers write custom sorting comparators using subtraction (e.g., `(a, b) -> a[1] - b[1]`).
- This is highly dangerous because coordinates range between -2^{31} and $2^{31} - 1$. Subtracting a large positive number from a large negative number will cause **integer overflow**, leading to incorrect sorting or crashes. Always use standard less-than comparisons (`a[1] < b[1]`).

15.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log n)$	Dominated by the sorting step. The linear scan takes $\mathcal{O}(n)$ time.
Space Complexity	$\mathcal{O}(\log n)$ or $\mathcal{O}(n)$	$\mathcal{O}(\log n)$ auxiliary space in C++ (Introsort stack), $\mathcal{O}(n)$ in Python (Timsort).

15.6 Common Follow-Up Questions & How to Answer

15.6.1 Q1: What is the difference between this problem and Non-overlapping Intervals (LeetCode 435)?

- **Answer**: The core logic is identical—both are variants of the **Interval Scheduling Problem**. However, the boundary overlap conditions differ:
  - In LeetCode 452 (Burst Balloons), intervals that touch at a single point (e.g., `[1, 2]` and `[2, 3]`) are considered **overlapping** (an arrow at $x = 2$ bursts both).
  - In LeetCode 435 (Non-overlapping Intervals), intervals touching at a single point (e.g., `[1, 2]` and `[2, 3]`) are **not** considered overlapping.

15.6.2 Q2: What if the coordinates are in a 2D plane and we shoot arrows diagonally?

- **Answer**: If we can shoot arrows along any line $y = mx + c$, the problem becomes equivalent to finding the minimum number of lines that intersect a set of geometric shapes. This is a much harder geometric optimization problem (typically NP-hard depending on constraints).

15.7 Pro-Tip: How to Impress the Interviewer

- **Highlight the Comparator Integer Overflow Risk**: Proactively warn the interviewer about comparator subtraction bugs (`a[1] - b[1]`). Mentioning how to safely sort intervals containing large negative and positive integers demonstrates high attention to code correctness and system safety.
- **Avoid Slicing in Python**: Pointing out that `points[1:]` allocates a new list and replacing it with an iterator demonstrates a strong grasp of Python's memory management.
- **In-Place Modification Communication**: Clarify with the interviewer if you are allowed to sort the input array `points` in-place. If modifying the input is forbidden, you must make a copy, which increases the space complexity to $\mathcal{O}(N)$.

16 Assign Cookies

- **Category**: Coding
- **Difficulty**: Easy
- **Target Role**: Software Engineer / QA & Test Engineer
- **Source**: LeetCode 455, Glassdoor

- **Flashcards**: Greedy deck

16.1 Question Description

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor `g[i]`, which is the minimum size of a cookie that the child will be content with; and each cookie j has a size `s[j]`. If `s[j] >= g[i]`, we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

16.1.1 Examples

- **Input**: `g = [1,2,3]`, `s = [1,1]`
  - **Output**: `1`
  - **Explanation**: You have 3 children and 2 cookies. The greed factors of the 3 children are 1, 2, 3. Even though you have 2 cookies, since their size is both 1, you can only make the child whose greed factor is 1 content.
- **Input**: `g = [1,2]`, `s = [1,2,3]`
  - **Output**: `2`
  - **Explanation**: You have 2 children and 3 cookies. The greed factors of the 2 children are 1, 2. You have 3 cookies and their sizes are big enough to gratify all children. Return 2.

16.2 Detailed Solution (C++)

To satisfy the maximum number of children, we should adopt a **Greedy Strategy**: 1. **Sort** both arrays: `g` (greed factors of children) and `s` (cookie sizes) in ascending order. 2. Use **Two Pointers** (`child` and `cookie`) initialized to `0`. 3. Try to satisfy the child with the smallest greed factor with the smallest possible cookie size. * If the current cookie size is sufficient (`s[cookie] >= g[child]`), the child is satisfied, and we move to the next child (`child++`). * Regardless of whether the child is satisfied or not, we must move to the next cookie (`cookie++`) because this cookie is either used or too small to satisfy the current child (and thus too small for any subsequent child with a larger greed factor). 4. When we run out of either children or cookies, `child` represents the number of content children.

16.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>

class Solution {
public:
    int findContentChildren(std::vector<int>& g, std::vector<int>& s) {
        // Sort greed factors and cookie sizes in-place
```

```
        std::sort(g.begin(), g.end());
        std::sort(s.begin(), s.end());

        int child = 0;
        int cookie = 0;
        const int num_children = static_cast<int>(g.size());
        const int num_cookies = static_cast<int>(s.size());

        // Greedily match cookies to children
        while (child < num_children && cookie < num_cookies) {
            if (s[cookie] >= g[child]) {
                child++; // Child is content, move to the next child
            }
            cookie++; // Move to the next cookie in all cases
        }

        return child;
    }
};
```

16.3 Detailed Solution (Python)

16.3.1 Standard Python Production Code

Below is the type-hinted, optimized Python implementation using a simplified `for` loop over cookies.

```
from typing import List

class Solution:
    def findContentChildren(self, g: List[int], s: List[int]) -> int:
        """
        Maximizes the number of content children given greed factors and cookie sizes.

        Time Complexity: O(N log N + M log M)
        Space Complexity: O(1) auxiliary (in-place sort)
        """
        # Sort in-place to minimize space allocations
        g.sort()
        s.sort()

        child = 0
        n_children = len(g)

        # Iterating through cookies directly avoids index checks inside the loop,
```

```
    # which is faster and more Pythonic.
    for cookie_size in s:
        if child < n_children and cookie_size >= g[child]:
            child += 1
        if child == n_children:
            break  # Early exit if all children are already satisfied

    return child
```

16.4 Python-Specific Complexities & Implementation Considerations

16.4.1 1. Loop Condition Optimization

- In Python, a standard `while child < len(g) and cookie < len(s):` loop involves multiple dictionary lookups and bounds checks at each step in the bytecode.
- By rewriting the loop to iterate directly over the elements of `s` (using `for cookie_size in s:`), we delegate index tracking to Python's internal C-iterator, speeding up execution.

16.4.2 2. In-Place Sorting vs. Creating Copies

- Using `g.sort()` and `s.sort()` modifies the input lists in-place, which is highly memory-efficient.
- Using `sorted(g)` and `sorted(s)` would allocate new list copies, causing $\mathcal{O}(N + M)$ space complexity. Always ask the interviewer if mutating the input parameters is acceptable.

16.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log n + m \log m)$	Where n is the number of children and m is the number of cookies. Sorting dominates the execution time.
Space Complexity	$\mathcal{O}(\log n + \log m)$ or $\mathcal{O}(n + m)$	Sorting stack space in C++ is $\mathcal{O}(\log n + \log m)$, while Python's Timsort uses $\mathcal{O}(n + m)$ auxiliary space.

16.6 Common Follow-Up Questions & How to Answer

16.6.1 Q1: What if children can be given more than one cookie (e.g., up to two cookies) to satisfy their greed factor?

- **Answer**: If a child can receive up to two cookies, we can no longer use the simple two-pointer greedy match. This is because a child's greed could be satisfied by a single large cookie or the sum of two smaller cookies.
  - To solve this greedily, we still sort children by greed factor.
  - We try to satisfy the child with the smallest greed factor first. We first check if the smallest single cookie can satisfy them. If not, we check if the sum of the two smallest cookies can satisfy them. If so, we assign them; otherwise, the child cannot be satisfied.

16.6.2 Q2: What if we can divide cookies? (e.g., a cookie of size 10 can be split into sizes 4 and 6)

- **Answer**: If cookies can be split arbitrarily, the individual cookie boundaries no longer matter. Only the total sum of cookie sizes matters. To maximize the number of content children, we should satisfy the children with the lowest greed factors first.
  1. Sort the children's greed factors `g`.
  2. Sum the cookie sizes to get `total_cookie_weight`.
  3. Iterate through `g` and subtract `g[child]` from `total_cookie_weight`.
  4. Stop when `total_cookie_weight` becomes negative. The index will represent the maximum satisfied children.

16.7 Pro-Tip: How to Impress the Interviewer

- **Mention Early Exit Optimization**: In the Python solution, adding `if child == n_children: break` allows the program to terminate immediately if we satisfy all children before we run out of cookies. In real-world scenarios with a large number of cookies, this saves significant processing time.
- **Discuss Constness and Pass-by-Reference in C++**: Show interviewers that you care about performance by passing vectors by reference. If the caller allows input mutation, sort them in-place. If not, explicitly mention that you must copy the vectors first, which shifts auxiliary space complexity to $\mathcal{O}(N + M)$.

17 IPO

- **Category**: Coding
- **Difficulty**: Hard
- **Target Role**: Software Engineer / AI Systems Architect
- **Source**: LeetCode 502, Glassdoor
- **Flashcards**: Greedy deck

17.1 Question Description

Suppose LeetCode will start its IPO soon. In order to sell a good price of its shares to Venture Capital, LeetCode would like to work on some projects to increase its capital before the IPO. Since it has limited resources, it can only finish at most k distinct projects before the IPO. Help LeetCode design the best way to maximize its total capital after finishing at most k distinct projects.

You are given n projects where the i-th project has a pure profit `profits[i]` and a minimum capital of `capital[i]` is needed to start it.

Initially, you have w capital. When you finish a project, you will obtain its pure profit and the profit will be added to your total capital.

Pick a list of at most k distinct projects from given projects to maximize your final capital, and return the final maximized capital.

The answer is guaranteed to fit in a 32-bit signed integer.

17.1.1 Examples

- **Input**: `k = 2`, `w = 0`, `profits = [1,2,3]`, `capital = [0,1,1]`
  - **Output**: `4`
  - **Explanation**:
    - Since your initial capital is 0, you can only start the project indexed 0.
    - After finishing it, you obtain profit 1, and your capital becomes 1.
    - With capital 1, you can either start the project indexed 1 or 2.
    - Since you can choose at most 2 projects, you should choose project 2 to get the maximum capital.
    - Total capital is $0 + 1 + 3 = 4$.
- **Input**: `k = 3`, `w = 0`, `profits = [1,2,3]`, `capital = [0,1,2]`
  - **Output**: `6`

17.2 Detailed Solution (C++)

The problem can be solved greedily using a **Min-Heap (or Sorting) + Max-Heap** combination: 1. **Sort Projects**: Combine `capital[i]` and `profits[i]` into pairs and sort them in ascending order of their required capital. This allows us to easily find which projects can be started given our current capital w. 2. **Track Available Projects**: Maintain a `max_heap` of profits. As our capital w increases, we scan through the sorted projects and push the profits of all newly affordable projects (i.e., where `capital[i] <= w`) into the `max_heap`. 3. **Greedy Choice**: Out of all affordable projects in the `max_heap`, we greedily choose the one with the maximum profit. We pop it, add its profit to our capital w, and repeat this step up to k times. 4. **Early Exit**: If at any step before k projects the `max_heap` becomes empty (meaning we cannot afford any new projects and have no projects left in our pool), we terminate early and return w.

17.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <queue>
#include <algorithm>

class Solution {
public:
    int findMaximizedCapital(int k, int w, std::vector<int>& profits, std::vector<int>& capital) {
        const int n = static_cast<int>(profits.size());

        // Pair projects as (capital, profit)
        std::vector<std::pair<int, int>> projects(n);
        for (int i = 0; i < n; ++i) {
            projects[i] = {capital[i], profits[i]};
        }

        // Sort projects by capital required (ascending)
        std::sort(projects.begin(), projects.end());

        // Max-heap to store profits of all currently affordable projects
        std::priority_queue<int> max_profit_heap;
        int idx = 0;

        for (int step = 0; step < k; ++step) {
            // Push all affordable projects into the max-heap
            while (idx < n && projects[idx].first <= w) {
                max_profit_heap.push(projects[idx].second);
                idx++;
            }

            // If we have no affordable projects, we cannot proceed further
            if (max_profit_heap.empty()) {
                break;
            }

            // Greedily pick the project with the highest profit
            w += max_profit_heap.top();
            max_profit_heap.pop();
        }

        return w;
    }
};
```

17.3 Detailed Solution (Python)

17.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using `heapq` and negating profits to simulate a max-heap.

```python
from typing import List
import heapq

class Solution:
    def findMaximizedCapital(self, k: int, w: int, profits: List[int], capital: List[int]) -> int:
        """
        Maximizes the total capital after selecting at most k projects.

        Time Complexity: O(N log N + k log N)
        Space Complexity: O(N) to store projects pair list
        """
        # Zip and sort by capital
        projects = sorted(zip(capital, profits))

        # Min-heap simulated as max-heap by negating profits
        max_heap: List[int] = []
        idx = 0
        n = len(projects)

        for _ in range(k):
            # Push profits of all affordable projects (capital <= w)
            while idx < n and projects[idx][0] <= w:
                # Store negated profit to turn python's min-heap into a max-heap
                heapq.heappush(max_heap, -projects[idx][1])
                idx += 1

            # If no projects can be undertaken, exit early
            if not max_heap:
                break

            # Retrieve the project with the maximum profit
            w += -heapq.heappop(max_heap)

        return w
```

17.4 Python-Specific Complexities & Implementation Considerations

17.4.1 1. Simulating Max-Heaps via Negation

- Python's built-in `heapq` module only supports **Min-Heaps**. To simulate a **Max-Heap**, we negate the values before pushing them (`heapq.heappush(heap, -value)`) and negate them again when popping (`-heapq.heappop(heap)`).
- Care must be taken with the sign conversion when working with objects or multiple fields (e.g. tuples). For example, a tuple `(-profit, capital)` will prioritize profit first (descending) and capital second (ascending).

17.4.2 2. Space Optimization using `zip`

- In Python, `zip(capital, profits)` creates a generator. Passing it to `sorted()` creates a new list of tuples, which is highly efficient because the underlying sorting routine runs in C.
- However, this does copy the arrays into a new structure, consuming $\mathcal{O}(N)$ space. If memory is extremely tight, we can index a list of indices and sort it instead.

17.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log n + k \log n)$	Sorting the n projects takes $\mathcal{O}(n \log n)$. In the worst case, we push and pop n projects from the heap, taking $\mathcal{O}((n+k) \log n)$ time.
Space Complexity	$\mathcal{O}(n)$	To store the combined capital-profit pairings and the heap elements.

17.6 Common Follow-Up Questions & How to Answer

17.6.1 Q1: What if projects can be done multiple times?

- **Answer**: If a project can be repeated infinitely, we don't need to track multiple projects in a heap. At any step, we just want to perform the single most profitable project that we can afford. In this case, we can use a Segment Tree or Binary Indexed Tree (BIT) to find the maximum profit element with `capital[i] <= w`, query/update it, or use dynamic programming if the state space is small.

17.6.2 Q2: What if capital is deducted (i.e. starting a project has a transaction cost that is not refunded)?

- **Answer**: In the current problem, capital w is non-decreasing since we only need `capital[i] <= w` as a threshold, and then we get `profits[i]` back. If capital is actually *spent* (i.e. w is decremented by

`capital[i]` and incremented by `profits[i]`), our capital can fluctuate up and down. This variation makes the problem equivalent to the Knapsack problem with ordering dependencies, which is **NP-hard** and requires Dynamic Programming or Backtracking with pruning.

17.7 Pro-Tip: How to Impress the Interviewer

- **Explain the Monotonicity of Capital**: Explain that the greedy choice is optimal *because* our capital w is monotonically non-decreasing. Since w never decreases, any project that becomes affordable in step t remains affordable in all future steps $t+1, t+2, \ldots$. This allows us to keep a single pointer `idx` moving forward through the sorted projects list, guaranteeing a total runtime of $\mathcal{O}(n \log n + k \log n)$ rather than $\mathcal{O}(n \cdot k)$.
- **Prevent Double Heap Pushes**: Highlight that each project is pushed into the `max_profit_heap` at most once by keeping the `idx` pointer state persistent across iterations of the main k-loop. This is a critical optimization that prevents the algorithm from degrading to $\mathcal{O}(n \cdot k)$.

18 Task Scheduler

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / AI Systems Architect
- **Source**: LeetCode 621, Glassdoor
- **Flashcards**: Greedy deck

18.1 Question Description

You are given an array of CPU tasks, each labeled with a letter from A to Z, and a cooldown interval n. Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of at least n intervals between two tasks with the same label.

Return the *minimum number of CPU intervals* required to complete all tasks.

18.1.1 Examples

- **Input**: `tasks = ["A","A","A","B","B","B"]`, `n = 2`
  - **Output**: `8`
  - **Explanation**: A possible optimal sequence is: `A -> B -> idle -> A -> B -> idle -> A -> B`. After completing task A, you must wait 2 intervals before doing A again. The same applies to task B.
- **Input**: `tasks = ["A","C","A","B","D","B"]`, `n = 1`
  - **Output**: `6`
  - **Explanation**: A possible sequence is: `A -> B -> C -> D -> A -> B`. No idle intervals are required.

- **Input**: `tasks = ["A","A","A","B","B","B"]`, `n = 3`
  - **Output**: `10`
  - **Explanation**: A possible sequence is: `A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B`.

18.2 Detailed Solution (C++)

The problem can be solved mathematically by focusing on the most frequent tasks.

18.2.1 Mathematical Formula Intuition

1. Let the highest frequency of any task be `max_freq`. Let A be one of these tasks.
2. To schedule A with a gap of at least n intervals, we create `max_freq - 1` partitions. Each partition must have a size of `n + 1` (containing A itself followed by n slots for other tasks or idle periods).
3. The last partition only needs to accommodate the final occurrences of the tasks that share the maximum frequency (`max_count`).
4. Thus, the minimum length of the layout is:

$$\text{Formula} = (\text{maxFreq} - 1) \times (n + 1) + \text{maxCount}$$

5. If we have more tasks than can fit in these empty slots, we can insert them into the slots without increasing the required length beyond `tasks.size()`. Since there are enough distinct tasks, the scheduling will always be possible without any idle slots.
6. Therefore, the minimum intervals required is:

$$\max(\text{tasks.size}(), (\text{maxFreq} - 1) \times (n + 1) + \text{maxCount})$$

18.2.2 Standard C++ Production Code

```cpp
#include <vector>
#include <string>
#include <algorithm>

class Solution {
public:
    int leastInterval(std::vector<char>& tasks, int n) {
        // Frequency array for uppercase English letters A-Z
        int freq[26] = {0};
        for (char c : tasks) {
            freq[c - 'A']++;
        }

        // Find the maximum frequency
        int max_freq = 0;
        for (int i = 0; i < 26; ++i) {
```

```
            if (freq[i] > max_freq) {
                max_freq = freq[i];
            }
        }

        // Count how many tasks have this maximum frequency
        int max_count = 0;
        for (int i = 0; i < 26; ++i) {
            if (freq[i] == max_freq) {
                max_count++;
            }
        }

        // Apply the mathematical formula
        int part_count = max_freq - 1;
        int part_length = n + 1;
        int min_intervals = part_count * part_length + max_count;

        // The answer is the maximum of the formula and the total number of tasks
        return std::max(static_cast<int>(tasks.size()), min_intervals);
    }
};
```

18.3 Detailed Solution (Python)

18.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using `collections.Counter`.

```
from typing import List
from collections import Counter

class Solution:
    def leastInterval(self, tasks: List[str], n: int) -> int:
        """
        Calculates the minimum intervals required to complete all tasks with cooldown n.

        Time Complexity: O(N) where N is the number of tasks
        Space Complexity: O(1) auxiliary (at most 26 unique task keys)
        """
        if not tasks:
            return 0

        freq = Counter(tasks)
```

```
        max_freq = max(freq.values())

        # Count how many tasks share the maximum frequency
        max_count = sum(1 for v in freq.values() if v == max_freq)

        formula = (max_freq - 1) * (n + 1) + max_count
        return max(len(tasks), formula)
```

18.4 Python-Specific Complexities & Implementation Considerations

18.4.1 1. Python Character Arithmetic

- Unlike C++ where we can map characters directly to indices via `c - 'A'`, Python does not support character subtraction.
- We must use `ord(c) - ord('A')` to convert characters to integer indices. Using `collections.Counter` avoids this character conversion overhead altogether by hashing the task string values directly.

18.4.2 2. Generator Expression Overhead vs List Comprehension

- `sum(1 for v in freq.values() if v == max_freq)` uses a generator expression to calculate the count.
- For small dataset sizes (e.g., at most 26 unique characters), a list comprehension like `sum([1 for v in freq.values() if v == max_freq])` can be slightly faster in Python due to lower overhead in creating the generator object, although a generator is more memory-friendly for massive datasets.

18.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Where n is the total number of tasks. We perform a single pass to count task frequencies.
Space Complexity	$\mathcal{O}(1)$	The alphabet size is fixed to 26 letters, requiring constant storage.

18.6 Common Follow-Up Questions & How to Answer

18.6.1 Q1: How would you construct and return the actual scheduled sequence of tasks?

- **Answer**: To construct the actual task sequence, we use a **Max-Heap** and a **Queue**:
  1. Count frequencies and push all task counts and characters into a max-heap as `(frequency, task_char)`.

2. Maintain a queue to store tasks that are on cooldown, keeping track of the time they become available again: `(remaining_frequency, task_char, next_available_time)`.
3. At each time step `t`:
   - Check if any task at the front of the cooldown queue is ready (`next_available_time <= t`). If so, pop it and push it back onto the max-heap.
   - If the max-heap is not empty, pop the most frequent task, append it to our result, decrement its frequency, and if its frequency is still > 0, push it to the cooldown queue with `next_available_time = t + n + 1`.
   - If the max-heap is empty, append `"idle"`.
   - Increment `t`.

18.6.2 Q2: What if tasks have precedence dependencies (e.g. Task B can only start after Task A finishes)?

- **Answer**: This is a DAG Scheduling problem with communication/cooldown delays. We would combine **Topological Sort (Kahn's Algorithm)** with the greedy scheduling approach. A task can only be pushed into the max-heap when all its incoming dependency tasks are completed and its cooldown constraint is satisfied.

18.7 Pro-Tip: How to Impress the Interviewer

- **Explain the Math-Greedy Duality**: Explain to the interviewer that while a simulation using a max-heap and queue is intuitive and necessary if we need to output the schedule itself, the math formula method runs in $\mathcal{O}(N)$ time and $\mathcal{O}(1)$ auxiliary space, compared to the simulation's $\mathcal{O}(TotalTime \cdot \log 26)$. Since $TotalTime$ can be much larger than N (due to n and idles), the math solution is infinitely faster for large cooldown values.
- **Discuss Alphabet Size Generality**: Mention that the solution generalizes to any alphabet size $|\Sigma|$ (not just 26). The time complexity remains $\mathcal{O}(N + |\Sigma|)$ and space complexity $\mathcal{O}(|\Sigma|)$.
- **SIMD Vectorization potential in C++**: Point out that the C++ loop to count frequencies can easily be vectorized by modern compilers since it operates on a contiguous block of data.

19 Task Scheduler with Multiple Machines

- **Category**: Coding
- **Difficulty**: Hard
- **Target Role**: Software Engineer / AI Systems Architect
- **Source**: LeetCode 621 (Extension), Glassdoor
- **Flashcards**: Greedy deck

19.1 Question Description

You are given an array of CPU tasks, each labeled with a letter from A to Z, a cooldown period n, and m identical parallel machines.

Each machine can execute at most one task per time interval. Tasks can be completed in any order across the machines. Cooldown is tracked **per-machine**: if machine M runs task A at time t, that machine M cannot run task A again until time $t + n + 1$. Other machines are unaffected and may run task A at any time (including simultaneously with M).

Return the *minimum number of time intervals* required to complete all tasks across all m machines.

19.1.1 Examples

- **Input**: `tasks = ["A","A","A","B","B","B"]`, `n = 2`, `m = 1`
  - **Output**: `8`
  - **Explanation**: This is the original LeetCode 621 problem.
    - `A -> B -> idle -> A -> B -> idle -> A -> B`
- **Input**: `tasks = ["A","A","A","B","B","B"]`, `n = 2`, `m = 2`
  - **Output**: `4`
  - **Explanation**: Cooldown is per-machine, so machines can swap tasks.
    - `t=0`: M1=A, M2=B
    - `t=1`: M1=B, M2=A (M1 never ran B, and M2 never ran A)
    - `t=2`: idle (both machines must wait cooldown)
    - `t=3`: M1=A, M2=B
- **Input**: `tasks = ["A","C","A","B","D","B"]`, `n = 1`, `m = 2`
  - **Output**: `3`
  - **Explanation**:
    - `t=0`: M1=A, M2=B
    - `t=1`: M1=C, M2=D
    - `t=2`: M1=A, M2=B

19.2 Detailed Solution (C++)

With multiple machines and per-machine cooldown constraints, we cannot use a simple single-variable mathematical formula. Instead, we must simulate the scheduling process using a **Greedy Simulation with Discrete-Event Time Skipping**:

1. **Greedy Strategy**: At each time interval t, we attempt to assign tasks to each of the m machines. For each machine i (from 0 to $m - 1$), we look for the highest-frequency remaining task that is not on cooldown on that specific machine i.
2. **State Tracking**:
   - We keep an array `freq` of size 26 representing the remaining count of each task.
   - We maintain a 2D matrix `cooldown[m][26]` where `cooldown[mi][t]` stores the next time step machine `mi` is allowed to execute task `t`.
3. **Time Skip (Discrete-Event Optimization)**: If in a time interval t we are unable to assign *any* task to *any* machine (meaning all remaining tasks are on cooldown on all machines), incrementing time by 1 repeatedly leads to wasteful "busy-waiting" loops. Instead, we jump `time` directly to the next earliest cooldown expiry time among all active cooldowns.

19.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>
#include <climits>

class Solution {
public:
    int leastInterval(std::vector<char>& tasks, int n, int m) {
        int freq[26] = {0};
        for (char c : tasks) {
            freq[c - 'A']++;
        }

        // cooldown[mi][task] stores the next time machine mi can run task
        // Initialize to INT_MIN (meaning immediately runnable)
        std::vector<std::vector<int>> cooldown(m, std::vector<int>(26, INT_MIN));

        int time = 0;
        int remaining = static_cast<int>(tasks.size());

        while (remaining > 0) {
            bool assigned = false;

            // Greedily assign tasks to each machine at the current time step
            for (int mi = 0; mi < m; ++mi) {
                int best_task = -1;
                int best_count = 0;

                for (int t = 0; t < 26; ++t) {
                    if (freq[t] > best_count && cooldown[mi][t] <= time) {
                        best_task = t;
                        best_count = freq[t];
                    }
                }

                if (best_task >= 0) {
                    freq[best_task]--;
                    cooldown[mi][best_task] = time + n + 1;
                    remaining--;
                    assigned = true;
                }
            }
```

```
            if (assigned) {
                time++;
            } else {
                // Time Jump: Find the next earliest time a cooldown expires
                int next_time = INT_MAX;
                for (int mi = 0; mi < m; ++mi) {
                    for (int t = 0; t < 26; ++t) {
                        if (cooldown[mi][t] > time && cooldown[mi][t] < next_time) {
                            next_time = cooldown[mi][t];
                        }
                    }
                }
                time = next_time;
            }
        }

        return time;
    }
};
```

19.3 Detailed Solution (Python)

19.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using a dense list of lists for optimal cooldown lookups.

```
from typing import List
from collections import Counter

class Solution:
    def leastInterval(self, tasks: List[str], n: int, m: int) -> int:
        """
        Calculates the minimum CPU intervals to execute tasks on m parallel machines
        with a per-machine cooldown constraint of n.

        Time Complexity: O(T * m * 26) where T is the total active time steps.
        Space Complexity: O(m * 26) for tracking cooldown states.
        """
        freq = Counter(tasks)

        # cooldown[mi][task_index] stores the next available time for machine mi
        cooldown = [[0] * 26 for _ in range(m)]

        time = 0
```

```
    remaining = len(tasks)

    # Map character tasks to 0-25 index
    def task_to_idx(char: str) -> int:
        return ord(char) - ord('A')

    while remaining > 0:
        assigned = False

        for i in range(m):
            best_task = None
            best_count = 0

            for task, count in freq.items():
                if count > best_count:
                    task_idx = task_to_idx(task)
                    if cooldown[i][task_idx] <= time:
                        best_task = task
                        best_count = count

            if best_task:
                freq[best_task] -= 1
                if freq[best_task] == 0:
                    del freq[best_task]  # Clean up dictionary

                task_idx = task_to_idx(best_task)
                cooldown[i][task_idx] = time + n + 1
                remaining -= 1
                assigned = True

        if assigned:
            time += 1
        else:
            # Fast forward to next cooldown expiry to prevent idle busy-loops
            next_time = min(
                t for machine_cooldowns in cooldown
                for t in machine_cooldowns if t > time
            )
            time = next_time

    return time
```

19.4 Python-Specific Complexities & Implementation Considerations

19.4.1 1. Dictionary Size Modification During Iteration

- In Python, we cannot delete keys from a dictionary while iterating over it (doing so raises a `RuntimeError: dictionary changed size during iteration`).
- In our code, we iterate over `freq.items()`. When a task count reaches 0, we delete it using `del freq[best_task]`. However, this deletion occurs *outside* the inner loop that iterates over `freq.items()`, making it safe.

19.4.2 2. Dense Matrix `cooldown` vs List of Dictionaries

- Using a dense 2D list `cooldown = [[0] * 26 for _ in range(m)]` is significantly faster than using a list of dicts `[{} for _ in range(m)]` because array indexing via integer offsets is much faster than hashing string keys inside nested loops.

19.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(T \cdot m \cdot \lvert\Sigma\rvert)$	Where T is the number of time intervals, m is the number of machines, and $\lvert\Sigma\rvert = 26$ is the task alphabet size.
Space Complexity	$\mathcal{O}(m \cdot \lvert\Sigma\rvert)$	Required to maintain the per-machine cooldown matrix.

19.6 Common Follow-Up Questions & How to Answer

19.6.1 Q1: What if the cooldown is global (shared across all machines) instead of per-machine?

- **Answer**: If the cooldown is global (e.g. if machine 1 runs A, no machine can run A for n units), the problem becomes equivalent to the single-machine task scheduler but with a faster consumption rate. We can model this by running m tasks per interval (if available and not on global cooldown) and scheduling them using a single global cooldown queue.

19.6.2 Q2: How does this simulation scale if the number of unique task types is very large (e.g. $\lvert\Sigma\rvert \geq 10^5$)?

- **Answer**: If $\lvert\Sigma\rvert$ is large, scanning all tasks linearly to find the `best_task` becomes too slow. We should replace the linear scan with a **Priority Queue (Max-Heap)** storing the remaining counts of tasks that are currently available.
  - We maintain a global max-heap of ready tasks.

  - We maintain a cooldown list/queue per machine.
  - At each step, we pop from the max-heap and assign it to the machine. If a machine cannot run the top task, we temporarily buffer the popped tasks until we find a task the machine is allowed to run, then push the buffer back into the heap.

19.7 Pro-Tip: How to Impress the Interviewer

- **Highlight the Discrete-Event Skipping Optimization**: In real-world systems engineering (e.g., job schedulers like Kubernetes, Mesos, or Yarn), time is not incremented unit-by-unit if all resources are blocked. Implementing the time-skip optimization (`time = next_time`) shows that you write production-ready, event-driven systems code rather than naive loops.
- **Discuss Cache Alignment of the Cooldown Matrix**: In C++, since the matrix dimensions are small ($m \leq 26$, $|\Sigma| = 26$), the entire `cooldown` matrix easily fits inside a single L1 cache line (which is typically 64 bytes for modern CPUs). Keeping the rows contiguous in memory ensures cache friendly access.

20 Largest Rectangle in Histogram

- **Category**: Coding
- **Difficulty**: Hard
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 84, Glassdoor
- **Flashcards**: Monotonic Stack deck

20.1 Question Description

Given an array of integers `heights` representing the histogram's bar height where the width of each bar is `1`, return the area of the largest rectangle in the histogram.

20.1.1 Examples

- **Input**: `heights = [2,1,5,6,2,3]`
  - **Output**: `10`
  - **Explanation**: The largest rectangle is formed by the bars `[5, 6]` with a height of `5` and a width of `2`, yielding an area of $5 \times 2 = 10$ units.
- **Input**: `heights = [2,4]`
  - **Output**: `4`
  - **Explanation**: The largest rectangle is formed by the bar `[4]` with a height of `4` and a width of `1` (or two bars of height `2` and width `2`), yielding an area of `4` units.

20.2 Detailed Solution (C++)

The optimal approach utilizes a **Monotonic Stack**. We maintain a stack that stores indices of the bars in a strictly increasing order of their heights.

When we encounter a bar that is shorter than the bar at the top of the stack, we know that the bar at the top of the stack cannot extend further to the right. We then pop the top elements from the stack and compute the area of the rectangles that can be formed using each popped bar as the height: 1. The **height** of the rectangle is the height of the popped bar. 2. The **width** of the rectangle is determined by the current index `i` (which is the right boundary) and the index of the new top element of the stack (which is the left boundary). If the stack becomes empty after popping, the width is simply `i`. 3. To avoid cleanup code after the loop, we process a virtual sentinel element of height `0` at index `n` (where `n` is the length of `heights`). This forces all remaining elements in the stack to be popped and processed.

20.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>
#include <cstddef>

class Solution {
public:
    int largestRectangleArea(std::vector<int>& heights) {
        // Edge Case: Explicitly handle empty input to prevent unsigned underflow
        if (heights.empty()) {
            return 0;
        }

        const int n = static_cast<int>(heights.size());
        std::vector<int> stack;
        stack.reserve(n); // Pre-allocate memory to prevent reallocation overhead
        int max_area = 0;

        for (int i = 0; i <= n; ++i) {
            // Use 0 as a sentinel value at i == n to flush the stack
            int current_height = (i < n) ? heights[i] : 0;

            while (!stack.empty() && current_height < heights[stack.back()]) {
                int height = heights[stack.back()];
                stack.pop_back();

                // If stack is empty, the popped bar was the shortest bar seen so far,
                // so it can span from index 0 to i - 1 (width = i)
                int width = stack.empty() ? i : i - stack.back() - 1;
                max_area = std::max(max_area, height * width);
```

```
            }
            stack.push_back(i);
        }

        return max_area;
    }
};
```

20.3 Detailed Solution (Python)

20.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using the monotonic stack with a sentinel value.

```python
from typing import List

class Solution:
    def largestRectangleArea(self, heights: List[int]) -> int:
        """
        Calculates the area of the largest rectangle in a histogram.

        Time Complexity: O(N)
        Space Complexity: O(N)
        """
        if not heights:
            return 0

        # Append a sentinel height 0 to flush out all indices from the stack at the end
        extended_heights = heights + [0]
        stack: List[int] = []
        max_area = 0

        for i, h in enumerate(extended_heights):
            while stack and h < extended_heights[stack[-1]]:
                height = extended_heights[stack.pop()]
                # If stack is empty, this bar was the smallest so far, so width is i.
                # Otherwise, the width is the distance between current index i and the
                # new top element on the stack minus 1.
                width = i if not stack else i - stack[-1] - 1
                max_area = max(max_area, height * width)
            stack.append(i)

        return max_area
```

20.4 Python-Specific Complexities & Implementation Considerations

20.4.1 1. Sentinel List Concatenation vs. Explicit Flushing

- In Python, `heights + [0]` creates a new list, which takes $\mathcal{O}(n)$ time and auxiliary space. While this is clean and concise, in memory-constrained environments or with extremely large inputs, it is better to avoid copying.
- Alternatively, we can iterate up to `len(heights)` and dynamically return `0` for the out-of-bound index to avoid creating a new list.

20.4.2 2. Efficiency of Stack Operations

- Python `list` acts as a dynamic array. Popping from the end via `pop()` and appending via `append()` are amortized $\mathcal{O}(1)$ operations.
- Avoid using `list.pop(0)` or `collections.deque` if only accessing the end, as `deque` has higher constant factor allocation overhead in Python compared to a pre-allocated/dynamic list.

20.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Each index is pushed onto the stack exactly once and popped at most once.
Space Complexity	$\mathcal{O}(n)$	The stack can store up to n elements in the worst case (e.g., when the heights are in strictly increasing order).

20.6 Common Follow-Up Questions & How to Answer

20.6.1 Q1: Can we solve the problem using a Divide and Conquer approach? What is its complexity?

- **Answer**: Yes, we can find the minimum bar in the current range, compute the area with that minimum bar as the height, and recursively solve for the left and right sub-histograms.
- **Complexity**: The recurrence is $T(n) = 2T(n/2) + \mathcal{O}(n)$ in the best case (when the minimum bar is in the middle), giving $\mathcal{O}(n \log n)$ time. However, in the worst case (already sorted heights), it degrades to $\mathcal{O}(n^2)$. We can optimize the range minimum query to $\mathcal{O}(\log n)$ using a **Segment Tree**, ensuring $\mathcal{O}(n \log n)$ time complexity in the worst case.

20.6.2 Q2: How is this problem related to "Maximal Rectangle" in a 2D binary matrix (LeetCode 85)?

- **Answer**: LeetCode 85 can be solved by reducing it to LeetCode 84. For each row in the 2D matrix, we can treat the number of consecutive `1`s above it as a histogram height. We can update these heights in $\mathcal{O}(\text{cols})$ time for each row, and then run the Largest Rectangle in Histogram algorithm on the current row's histogram heights. Overall time complexity becomes $\mathcal{O}(\text{rows} \times \text{cols})$ instead of the naive $\mathcal{O}(\text{rows}^3 \times \text{cols}^3)$.

20.6.3 Q3: What if the widths of the bars are not 1, but are given in another array `widths`?

- **Answer**: If we have an array `widths` where `widths[i]` represents the width of the i-th bar, the prefix sum of widths can be precomputed. Specifically, we define `prefix_widths[i]` as the sum of widths of all bars up to $i-1$. Then, when popped, the width of the rectangle is `prefix_widths[i] - prefix_widths[stack.back()]` (or `prefix_widths[i]` if the stack is empty).

20.7 Pro-Tip: How to Impress the Interviewer

- **Catch the Vector Size Underflow**: In C++, always check if `heights` is empty before performing operations like `heights.size() - 1` to prevent underflow of unsigned `size_t`.
- **Stack Memory Pre-allocation**: Mentioning `stack.reserve(n)` in C++ shows optimization awareness. It prevents multiple allocations/deallocations as the vector grows dynamically.
- **Real-world Analogy (Skyline Queries)**: Explain that monotonic stack patterns are widely used in databases and GIS systems for computing 2D skyline queries, visibility polygons, and terrain rendering.

21 132 Pattern

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 456, Glassdoor
- **Flashcards**: Monotonic Stack deck

21.1 Question Description

Given an array of n integers `nums`, a **132 pattern** is a subsequence of three integers `nums[i]`, `nums[j]` and `nums[k]` such that `i < j < k` and `nums[i] < nums[k] < nums[j]`.

Return `true` if there is a **132 pattern** in `nums`, otherwise, return `false`.

21.1.1 Examples

- **Input**: `nums = [1,2,3,4]`

  - **Output**: `false`
  - **Explanation**: There is no 132 pattern in the sequence.
- **Input**: `nums = [3,1,4,2]`
  - **Output**: `true`
  - **Explanation**: There is a 132 pattern in the sequence: `[1, 4, 2]` (at indices 1, 2, and 3).
- **Input**: `nums = [-1,3,2,0]`
  - **Output**: `true`
  - **Explanation**: There are multiple 132 patterns, such as `[-1, 3, 2]`, `[-1, 3, 0]`, and `[-1, 2, 0]`.

21.2 Detailed Solution (C++)

To solve this problem in $\mathcal{O}(n)$ time, we can iterate from **right to left** (from index $n-1$ down to 0) and maintain a monotonic stack of candidate values for `nums[k]` (the "2" in the 132 pattern).

We keep track of a variable `third` (representing `nums[k]`), initialized to `INT_MIN` / `LONG_MIN`. 1. If we find a `nums[i]` (the "1" in the 132 pattern) such that `nums[i] < third`, we immediately return `true`. Since `third` represents a valid `nums[k]` that is smaller than some `nums[j]` to its left, and now `nums[i]` is further to the left and smaller than `nums[k]`, the pattern `nums[i] < nums[k] < nums[j]` is satisfied. 2. While the stack is not empty and the current element `nums[i]` is greater than the stack's top element, the stack top becomes a candidate for `third`. We pop the stack and update `third` to be this popped value. 3. We then push `nums[i]` onto the stack.

By popping elements smaller than `nums[i]`, we ensure that `third` always holds the largest possible value of `nums[k]` that is smaller than some element to its left. This maximizes the likelihood of satisfying `nums[i] < third` later in the iteration.

21.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <climits>
#include <algorithm>

class Solution {
public:
    bool find132pattern(std::vector<int>& nums) {
        // Edge Case: 132 pattern requires at least 3 elements
        if (nums.size() < 3) {
            return false;
        }

        // Use long to prevent integer overflow/underflow comparisons
        long third = LONG_MIN;
        std::vector<int> stack;
        stack.reserve(nums.size());
```

```
        // Traverse from right to left
        for (int i = static_cast<int>(nums.size()) - 1; i >= 0; --i) {
            // Step 1: Check if nums[i] fits the "1" in "132"
            if (static_cast<long>(nums[i]) < third) {
                return true;
            }

            // Step 2: Maintain monotonic stack. If nums[i] is greater than top,
            // the top becomes the new candidate for "2" (third).
            while (!stack.empty() && stack.back() < nums[i]) {
                third = static_cast<long>(stack.back());
                stack.pop_back();
            }

            // Step 3: Push nums[i] as a candidate for "3"
            stack.push_back(nums[i]);
        }

        return false;
    }
};
```

21.3 Detailed Solution (Python)

21.3.1 Standard Python Production Code

Below is the iterative Python implementation traversing from right to left.

```
from typing import List

class Solution:
    def find132pattern(self, nums: List[int]) -> bool:
        """
        Determines if there is a 132 pattern in nums.

        Time Complexity: O(N)
        Space Complexity: O(N)
        """
        if len(nums) < 3:
            return False

        # 'third' stores the value of the '2' in the '132' pattern (nums[k])
        third = float("-inf")
```

```
        stack: List[int] = []

        # Traverse the list in reverse order
        for num in reversed(nums):
            # If the current number is less than the largest available '2',
            # we have found a valid '1' (num < third)
            if num < third:
                return True

            # If current number is greater than the stack's top,
            # pop from the stack and update 'third'
            while stack and stack[-1] < num:
                third = stack.pop()

            stack.append(num)

        return False
```

21.4 Python-Specific Complexities & Implementation Considerations

21.4.1 1. Representation of Negative Infinity

- In Python, `float("-inf")` is the standard way to represent negative infinity. Comparison operations like `num < third` work correctly with `float("-inf")` across both integer and floating-point types.
- Avoid using magic numbers like `-10**9 - 1` as bounds, since constraints might change or be exceeded in custom environments.

21.4.2 2. Space Optimization via In-Place Stack (Conceptual)

- In Python, lists are dynamic arrays. Modifying the list in-place while iterating is generally discouraged unless memory constraints are critical. If requested, Python's list slice or indexing can mimic an in-place stack, but standard list operations are cleaner and highly optimized.

21.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Each element is pushed to and popped from the stack at most once.

Metric	Complexity	Description
Space Complexity	$\mathcal{O}(n)$	The stack stores up to n elements in the worst case (e.g., when the array is sorted in ascending order).

21.6 Common Follow-Up Questions & How to Answer

21.6.1 Q1: Can we solve this problem using a Left-to-Right traversal?

- **Answer**: Yes, we can precompute the prefix minimums of the array. Let `min_array[i]` be the minimum value from `nums[0]` to `nums[i]`. `min_array[i]` represents the best candidate for `nums[i]` (the "1").
- Then, traversing from right to left (index j), we maintain a stack of candidates for `nums[k]` (the "2"). For each `nums[j]`, we pop all elements from the stack that are less than or equal to `min_array[j]` (since they can't be greater than the prefix minimum). If the top of the stack is less than `nums[j]`, we return `true` (since it is also greater than `min_array[j]`). Otherwise, we push `nums[j]` to the stack.

21.6.2 Q2: Can we optimize the space complexity to $\mathcal{O}(1)$?

- **Answer**: Yes, if we are allowed to modify the input array, we can use the suffix of the array as the stack itself. Since the stack size is always less than or equal to the number of elements processed, we can use a write pointer starting from the end of the array to overwrite processed values and act as a stack.
- **Code Example (C++)**: `cpp bool find132pattern(std::vector<int>& nums) { int n = nums.size(); int top = n; // Index of top of the stack int third = INT_MIN; for (int i = n - 1; i >= 0; i--) { if (nums[i] < third) return true; while (top < n && nums[i] > nums[top]) { third = nums[top++]; } top--; nums[top] = nums[i]; // Use input array as stack } return false; }`

21.7 Pro-Tip: How to Impress the Interviewer

- **Propose the In-Place Stack Optimization Proactively**: Mention that the monotonic stack space can be reduced to $\mathcal{O}(1)$ auxiliary space if we are allowed to mutate the input array, using the array itself to store stack values.
- **Explain the Logical Duality**: Contrast the right-to-left traversal (tracking the largest potential `third`) with the left-to-right prefix-min traversal. Explaining both approaches demonstrates a deep understanding of monotonic stacks and search state tracking.

22 Next Greater Element II

- **Category**: Coding

- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 503, Glassdoor
- **Flashcards**: Monotonic Stack deck

22.1 Question Description

Given a circular integer array `nums` (i.e., the next element of `nums[nums.length - 1]` is `nums[0]`), return *the next greater number for every element in* `nums`.

The **next greater number** of a number `x` is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return `-1` for this number.

22.1.1 Examples

- **Input**: `nums = [1,2,1]`
  - **Output**: `[2,-1,2]`
  - **Explanation**:
    - The first `1`'s next greater number is `2`.
    - The number `2` has no greater number.
    - The second `1`'s next greater number is found circularly, which is also `2`.
- **Input**: `nums = [1,2,3,4,3]`
  - **Output**: `[2,3,4,-1,4]`

22.2 Detailed Solution (C++)

To handle the circular property, we can simulate an array duplicated to double its size (`2 * n` elements) without allocating additional memory.

We maintain a **Monotonic Decreasing Stack** that stores the indices of elements: 1. We loop from `i = 0` to `2 * n - 1`. Let the current circular index be `idx = i % n`. 2. While the stack is not empty and the current element `nums[idx]` is greater than `nums[stack.back()]`, it means `nums[idx]` is the next greater element for the index at `stack.back()`. We pop the top index from the stack and store `nums[idx]` in `result[popped_index]`. 3. In the first pass (`i < n`), we push `idx` onto the stack. We do not push in the second pass since any element requiring a next greater element has already been placed in the stack during the first pass.

22.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <cstddef>

class Solution {
```

```cpp
public:
    std::vector<int> nextGreaterElements(std::vector<int>& nums) {
        // Edge Case: Handle empty input
        if (nums.empty()) {
            return {};
        }

        const int n = static_cast<int>(nums.size());
        std::vector<int> result(n, -1);
        std::vector<int> stack;
        stack.reserve(n); // Pre-allocate stack to avoid reallocation overhead

        // Traverse the array twice to simulate the circular array
        for (int i = 0; i < 2 * n; ++i) {
            int idx = i % n;

            // Maintain the monotonic decreasing stack
            while (!stack.empty() && nums[stack.back()] < nums[idx]) {
                result[stack.back()] = nums[idx];
                stack.pop_back();
            }

            // Only push indices to stack in the first pass
            if (i < n) {
                stack.push_back(idx);
            }
        }

        return result;
    }
};
```

22.3 Detailed Solution (Python)

22.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using the two-pass monotonic stack.

```python
from typing import List

class Solution:
    def nextGreaterElements(self, nums: List[int]) -> List[int]:
        """
        Finds the next greater element for each index in a circular array.
```

```
    Time Complexity: O(N)
    Space Complexity: O(N)
    """
    if not nums:
        return []

    n = len(nums)
    result = [-1] * n
    stack: List[int] = []

    # Loop 2 * n times to simulate circular traversal
    for i in range(2 * n):
        idx = i % n

        # Pop elements from stack if the current element is greater
        while stack and nums[stack[-1]] < nums[idx]:
            result[stack.pop()] = nums[idx]

        # Only push indices from the first pass
        if i < n:
            stack.append(idx)

    return result
```

22.4 Python-Specific Complexities & Implementation Considerations

22.4.1 1. Index Arithmetic and Modulo Operator

- In Python, `i % n` works identically to lower-level languages. However, modulo arithmetic incurs minor CPU interpreter overhead.
- For performance-critical code on large datasets, you can write two separate loops: one for `i` from `0` to `n-1` (where you push to the stack), and a second for `i` from `0` to `n-1` (where you only pop and update without pushing). This eliminates the modulo operator entirely.

22.4.2 2. Pre-allocation of Python Lists

- Initiating `result = [-1] * n` is faster and uses less memory than calling `.append()` iteratively in Python, because it pre-allocates the exact memory needed in a single C-level allocation.

22.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	We iterate $2n$ times. Each index is pushed onto the stack at most once and popped at most once.
Space Complexity	$\mathcal{O}(n)$	The stack can store up to n elements in the worst case (e.g., when the array is sorted in descending order).

22.6 Common Follow-Up Questions & How to Answer

22.6.1 Q1: Why do we only push indices to the stack when `i < n`?

- **Answer**: Pushing to the stack during the second pass (`i >= n`) is redundant. The second pass is only performed to resolve elements that were pushed during the first pass but did not find a next greater element to their right. Any element visited in the second pass has already had its own opportunity to be pushed and resolved during the first pass.

22.6.2 Q2: Can we solve this problem by traversing from right to left?

- **Answer**: Yes, we can traverse from $2n - 1$ down to 0. We maintain a monotonic stack of values (or indices) in descending order. For each element `nums[idx]` (where `idx = i % n`), we pop elements from the stack that are less than or equal to `nums[idx]`. If the stack is not empty, its top element is the next greater element. We record this in our result vector (only when `i < n`), and then push `nums[idx]` onto the stack.
- **Code Example (C++)**: `cpp std::vector<int> nextGreaterElements(std::vector<int>& nums) { int n = nums.size(); std::vector<int> result(n, -1); std::vector<int> stack; for (int i = 2 * n - 1; i >= 0; --i) { int idx = i % n; while (!stack.empty() && stack.back() <= nums[idx]) { stack.pop_back(); } if (i < n) { result[idx] = stack.empty() ? -1 : stack.back(); } stack.push_back(nums[idx]); } return result; }`

22.7 Pro-Tip: How to Impress the Interviewer

- **Propose Modulo Elimination**: Mention that replacing the `i % n` operator with two distinct loops (first pass with push/pop, second pass with pop-only) is a cache-friendly and instruction-friendly optimization that avoids the hardware division instruction required for modulo.
- **Memory Safety & Pre-sizing**: Emphasize using `reserve(n)` on the stack vector. In production C++, dynamic reallocation of std::vector triggers memory copies, which degrades execution speed on large datasets.

23 Daily Temperatures

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 739, Glassdoor
- **Flashcards**: Monotonic Stack deck

23.1 Question Description

Given an array of integers `temperatures` representing the daily temperatures, return an array `answer` such that `answer[i]` is the number of days you have to wait after the i-th day to get a warmer temperature. If there is no future day for which this is possible, keep `answer[i] == 0` instead.

23.1.1 Examples

- **Input**: `temperatures = [73,74,75,71,69,72,76,73]`
  - **Output**: `[1,1,4,2,1,1,0,0]`
- **Input**: `temperatures = [30,40,50,60]`
  - **Output**: `[1,1,1,0]`
- **Input**: `temperatures = [30,60,90]`
  - **Output**: `[1,1,0]`

23.2 Detailed Solution (C++)

The standard solution uses a **Monotonic Decreasing Stack** storing indices of the temperatures. 1. We iterate through the array from left to right. 2. For each day `i` with temperature `temperatures[i]`, we compare it with the temperature at the index stored at the top of the stack. 3. While the stack is not empty and the current temperature `temperatures[i]` is strictly greater than the temperature at the index on top of the stack (`temperatures[stack.back()]`): * We pop the top index `j`. * The number of days we had to wait for index `j` is `i - j`. * We set `answer[j] = i - j`. 4. We push the current index `i` onto the stack.

By the end of the iteration, any indices remaining on the stack do not have a warmer day in the future, and their default value `0` remains in `answer`.

23.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <cstddef>

class Solution {
public:
    std::vector<int> dailyTemperatures(std::vector<int>& temperatures) {
```

```
        if (temperatures.empty()) {
            return {};
        }

        const int n = static_cast<int>(temperatures.size());
        std::vector<int> answer(n, 0);
        std::vector<int> stack;
        stack.reserve(n); // Reserve memory to prevent dynamic reallocation overhead

        for (int i = 0; i < n; ++i) {
            // Keep popping elements from the stack while we find a warmer temperature
            while (!stack.empty() && temperatures[i] > temperatures[stack.back()]) {
                int prev_index = stack.back();
                stack.pop_back();
                answer[prev_index] = i - prev_index;
            }
            stack.push_back(i);
        }

        return answer;
    }
};
```

23.3 Detailed Solution (Python)

23.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using the monotonic stack.

```
from typing import List

class Solution:
    def dailyTemperatures(self, temperatures: List[int]) -> List[int]:
        """
        Calculates the number of days to wait for a warmer temperature.

        Time Complexity: O(N)
        Space Complexity: O(N)
        """
        if not temperatures:
            return []

        n = len(temperatures)
        answer = [0] * n
```

```
    stack: List[int] = []

    for i, temp in enumerate(temperatures):
        # While stack is not empty and current temperature is warmer than top of stack
        while stack and temp > temperatures[stack[-1]]:
            prev_index = stack.pop()
            answer[prev_index] = i - prev_index
        stack.append(i)

    return answer
```

23.4 Python-Specific Complexities & Implementation Considerations

23.4.1 1. Dynamic Memory Overhead

- In Python, lists grow dynamically. The stack can hold up to n indices, resulting in memory allocation and copying at the C-level under the hood. While Python handles this efficiently, using an in-place array pointer jumping method (detailed below) can avoid stack allocation entirely.

23.4.2 2. Fast Enumeration

- Using `enumerate(temperatures)` is faster and more Pythonic than looping over `range(len(temperatures))` and accessing elements by index, as it avoids index-lookup overhead inside the virtual machine interpreter loop.

23.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Each index is pushed and popped from the stack at most once.
Space Complexity	$\mathcal{O}(n)$	The stack can store up to n elements in the worst case (e.g., when temperatures are strictly decreasing).

23.6 Common Follow-Up Questions & How to Answer

23.6.1 Q1: Can we solve this problem in $\mathcal{O}(1)$ auxiliary space (excluding the output array)?

- **Answer**: Yes! By iterating from **right to left** and using the information already stored in the `answer` array to jump over colder days, we can avoid allocating an external stack.

- For each index `i` from $n-2$ down to 0, we set `curr = i + 1`.
- If `temperatures[curr] <= temperatures[i]`, then if `answer[curr] == 0`, it means there is no warmer day to the right of `curr`, hence there is no warmer day for `i`. If `answer[curr] > 0`, we can jump `curr += answer[curr]` to the next warmer day and repeat the comparison.
- **Code Example (C++)**: `cpp std::vector<int> dailyTemperatures(std::vector<int>& temperatures) { int n = temperatures.size(); std::vector<int> answer(n, 0); for (int i = n - 2; i >= 0; --i) { int curr = i + 1; while (curr < n && temperatures[curr] <= temperatures[i]) { if (answer[curr] == 0) { curr = n; // Break, no warmer day exists } else { curr += answer[curr]; // Jump to next warmer day } } if (curr < n) { answer[i] = curr - i; } } return answer; }`

23.6.2 Q2: How can we leverage the temperature constraint $30 \leq$ temperatures$[i] \leq 100$?

- **Answer**: Since the temperature range is extremely small (only 71 unique values), we can keep an array `next_occurrence` of size 101, initialized to ∞, which stores the earliest index where each temperature occurs.
- Iterating from right to left, for each day `i`, we look up the values in `next_occurrence` for all temperatures from `temperatures[i] + 1` to `100`. The minimum index found tells us the closest warmer day. We then update `next_occurrence[temperatures[i]] = i`. This takes $\mathcal{O}(71 \cdot n)$ time and $\mathcal{O}(1)$ auxiliary space.

23.7 Pro-Tip: How to Impress the Interviewer

- **Propose the Array-Jumping Optimization**: Point out that the space complexity can be reduced from $\mathcal{O}(n)$ to $\mathcal{O}(1)$ auxiliary space (excluding the output array) using the right-to-left array jumping technique. This shows exceptional problem-solving depth.
- **Discuss Cache Locality**: Note that the array-jumping algorithm exhibits superior cache locality compared to stack-based approaches because it reuses the output array and minimizes pointer jumps, which fits beautifully in low-level CPU caches.

24 Car Fleet

- **Category**: Coding
- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 853, Glassdoor
- **Flashcards**: Monotonic Stack deck

24.1 Question Description

There are `n` cars at given miles away from the starting mile `0`, traveling to reach the mile `target`.

You are given two integer arrays `position` and `speed`, both of length `n`, where `position[i]` is the starting mile of the i-th car and `speed[i]` is the speed of the i-th car in miles per hour.

A car cannot pass another car, but it can catch up and then travel next to it at the speed of the slower car. A **car fleet** is a car or cars driving next to each other. The speed of the car fleet is the minimum speed of any car in the fleet.

If a car catches up to a car fleet at the mile target, it will still be considered as part of the car fleet.

Return *the number of car fleets that will arrive at the destination.*

24.1.1 Examples

- **Input**: `target = 12`, `position = [10,8,0,5,3]`, `speed = [2,4,1,1,3]`
  - **Output**: `3`
  - **Explanation**:
    - The cars starting at `10` (speed `2`) and `8` (speed `4`) become a fleet, meeting at mile `12`. Arrival time: `(12 - 10) / 2 = 1` hour and `(12 - 8) / 4 = 1` hour.
    - The car starting at `0` (speed `1`) travels alone. Arrival time: `(12 - 0) / 1 = 12` hours.
    - The cars starting at `5` (speed `1`) and `3` (speed `3`) become a fleet, meeting at mile `6`. They travel together at speed `1` and arrive at `12` in `7` hours.
    - In total, we have `3` fleets.
- **Input**: `target = 10`, `position = [3]`, `speed = [3]`
  - **Output**: `1`
- **Input**: `target = 100`, `position = [0,2,4]`, `speed = [4,2,1]`
  - **Output**: `1`

24.2 Detailed Solution (C++)

The core algorithm is based on **sorting** and **greedy state tracking** (which conceptually simplifies a Monotonic Stack): 1. First, we compute the time it takes for each car to reach the destination: `time = (target - position) / speed`. 2. We sort the cars by their starting position in **descending order** (from closest to the target to farthest). 3. We iterate through the sorted list of cars: * The first car (closest to the target) will always arrive first or lead the first fleet. Its calculated time is the initial `slowest_time`. * For each subsequent car, if its calculated time is **less than or equal to** `slowest_time`, it means this car starts further back but would arrive earlier or at the same time if unobstructed. Since it cannot pass the cars ahead of it, it will collide/join with the fleet ahead. It does not form a new fleet. * If its calculated time is **greater than** `slowest_time`, it cannot catch up to the fleet ahead. It must form its own new fleet, and we update `slowest_time` to its calculated time. 4. We increment our fleet count every time a new fleet is formed.

24.2.1 Standard C++ Production Code

```cpp
#include <vector>
#include <algorithm>
#include <numeric>
```

```cpp
#include <cstddef>

class Solution {
public:
    int carFleet(int target, std::vector<int>& position, std::vector<int>& speed) {
        const int n = static_cast<int>(position.size());
        if (n <= 1) {
            return n;
        }

        // Create an index array and sort indices based on position in descending order
        std::vector<int> indices(n);
        std::iota(indices.begin(), indices.end(), 0);
        std::sort(indices.begin(), indices.end(), [&](int a, int b) {
            return position[a] > position[b];
        });

        int fleets = 0;
        double slowest_time = -1.0;

        for (int i = 0; i < n; ++i) {
            int idx = indices[i];
            // Calculate time using double to prevent division truncation
            double time = static_cast<double>(target - position[idx]) / speed[idx];

            // If this car takes longer than the fleet in front of it, it forms a new fleet
            if (time > slowest_time) {
                fleets++;
                slowest_time = time;
            }
        }

        return fleets;
    }
};
```

24.3 Detailed Solution (Python)

24.3.1 Standard Python Production Code

Below is the type-hinted Python implementation. It pairs positions and speeds, sorts them, and iterates to count the fleets.

```python
from typing import List

class Solution:
    def carFleet(self, target: int, position: List[int], speed: List[int]) -> int:
        """
        Calculates the number of car fleets that arrive at the target.

        Time Complexity: O(N log N)
        Space Complexity: O(N)
        """
        if not position:
            return 0

        # Pair position and speed, then sort by position in descending order
        cars = sorted(zip(position, speed), key=lambda x: x[0], reverse=True)

        fleets = 0
        slowest_time = -1.0

        for pos, spd in cars:
            time = (target - pos) / spd

            # If current car takes more time than the slowest car in the fleet ahead,
            # it cannot catch up and forms a new fleet.
            if time > slowest_time:
                fleets += 1
                slowest_time = time

        return fleets
```

24.4 Python-Specific Complexities & Implementation Considerations

24.4.1 1. Float Precision and Division

- Python 3's `/` operator performs float division naturally. However, floating-point numbers can suffer from precision limits. Since target and speed are up to 10^6, standard floats are more than sufficient.
- Sorting via `zip` is highly optimized in Python as it executes via compiled C routines under the hood.

24.4.2 2. Space Optimization

- Sorting `zip(position, speed)` creates a list of tuples, occupying $\mathcal{O}(n)$ auxiliary memory. To conserve memory on massive inputs, sorting indices or using in-place sort on custom objects is preferred.

24.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n \log n)$	The time complexity is dominated by sorting the cars by their starting position. The subsequent linear scan takes $\mathcal{O}(n)$ time.
Space Complexity	$\mathcal{O}(n)$	Sorting indices or zipping positions and speeds requires $\mathcal{O}(n)$ extra space.

24.6 Common Follow-Up Questions & How to Answer

24.6.1 Q1: What if we want to know the size of each fleet?

- **Answer**: We can track this by keeping a stack of fleet leaders or sizes. When a car starts a new fleet (i.e. `time > slowest_time`), we push a new fleet with size `1`. If a car joins the existing fleet (`time <= slowest_time`), we increment the size of the top fleet on the stack.

24.6.2 Q2: What is the relationship between this problem and LeetCode 1776 (Car Fleet II)?

- **Answer**: In Car Fleet II, cars travel infinitely, and we need to output the exact time each car collides with the car ahead of it. This requires a full **Monotonic Stack** to keep track of collision times.
- As we iterate from right to left, we compare the current car's collision time with the car in front of it. If the current car is faster than the one ahead, it will collide. If the collision happens after the car ahead has already collided with its front car, we must pop the stack and evaluate collisions with the next car in front.

24.7 Pro-Tip: How to Impress the Interviewer

- **Use Float Division Safely**: Show your awareness of truncation bugs in C++. Doing `(target - position[idx]) / speed[idx]` without casting to `double` performs integer division, which truncates fractional parts and leads to incorrect fleet merges.
- **Demonstrate Stack Reduction**: Point out that although the problem is labeled "stack", we can reduce space complexity by only tracking the `slowest_time` variable because we only care about the front car of the current fleet. This avoids stack allocation.

25 Sum of Subarray Minimums

- **Category**: Coding

- **Difficulty**: Medium
- **Target Role**: Software Engineer / Low-Level Developer / AI Systems Architect
- **Source**: LeetCode 907, Glassdoor
- **Flashcards**: Monotonic Stack deck

25.1 Question Description

Given an array of integers `arr`, find the sum of `min(b)`, where `b` ranges over every (contiguous) subarray of `arr`. Since the answer may be large, return the answer **modulo** $10^9 + 7$.

25.1.1 Examples

- **Input**: `arr = [3,1,2,4]`
  - **Output**: `17`
  - **Explanation**:
    - Subarrays are `[3]`, `[1]`, `[2]`, `[4]`, `[3,1]`, `[1,2]`, `[2,4]`, `[3,1,2]`, `[1,2,4]`, `[3,1,2,4]`.
    - Minimums are `3, 1, 2, 4, 1, 1, 2, 1, 1, 1`.
    - Sum of minimums is `17`.
- **Input**: `arr = [11,81,94,43,3]`
  - **Output**: `444`

25.2 Detailed Solution (C++)

To solve this problem in linear time, we compute the contribution of each element `arr[i]` to the final sum. An element `arr[i]` acts as the minimum for any subarray that: 1. Starts after the **previous smaller element** on its left (index `left[i]`). 2. Ends before the **next smaller element** on its right (index `right[i]`).

Thus, the number of choices for the left boundary is `i - left[i]` and the number of choices for the right boundary is `right[i] - i`. The total number of subarrays where `arr[i]` is the minimum is `(i - left[i]) * (right[i] - i)`.

To avoid double-counting subarrays when there are duplicate elements, we define strict inequality on one side and non-strict inequality on the other: * On the left: we find the index of the first element that is **strictly smaller or equal** (`arr[left[i]] <= arr[i]`). * On the right: we find the index of the first element that is **strictly smaller** (`arr[right[i]] < arr[i]`).

We can find both the `left` and `right` boundary indices in $\mathcal{O}(n)$ time using a Monotonic Stack.

25.2.1 Standard C++ Production Code (Single-Pass with Sentinel)

Instead of using two separate arrays and passes, we can compute the sum in a **single pass** using a sentinel value of `0` (since `arr[i] >= 1`).

```
#include <vector>
#include <cstddef>
```

```cpp
class Solution {
public:
    int sumSubarrayMins(std::vector<int>& arr) {
        const int n = static_cast<int>(arr.size());
        const long long MOD = 1000000007LL;

        std::vector<int> stack;
        stack.reserve(n + 1);
        long long total_sum = 0;

        // Iterate up to n to process the sentinel element (0) at index n
        for (int i = 0; i <= n; ++i) {
            // Use 0 as a sentinel to force-flush all remaining elements in the stack
            int current_val = (i == n) ? 0 : arr[i];

            while (!stack.empty() && arr[stack.back()] >= current_val) {
                int mid = stack.back();
                stack.pop_back();

                // Find left smaller boundary index (top of stack after popping)
                int left_boundary = stack.empty() ? -1 : stack.back();

                // Count of subarrays where arr[mid] is the minimum
                long long count_left = mid - left_boundary;
                long long count_right = i - mid;

                total_sum = (total_sum + static_cast<long long>(arr[mid]) * count_left * count_right) % MOD;
            }
            stack.push_back(i);
        }

        return static_cast<int>(total_sum);
    }
};
```

25.3 Detailed Solution (Python)

25.3.1 Standard Python Production Code

Below is the type-hinted Python implementation using the single-pass sentinel approach.

```python
from typing import List
```

```python
class Solution:
    def sumSubarrayMins(self, arr: List[int]) -> int:
        """
        Calculates the sum of minimums of all contiguous subarrays.

        Time Complexity: O(N)
        Space Complexity: O(N)
        """
        MOD = 10**9 + 7
        n = len(arr)
        total_sum = 0
        stack: List[int] = []

        # We append a sentinel 0 to the end of the array to flush the stack
        extended_arr = arr + [0]

        for i, val in enumerate(extended_arr):
            while stack and extended_arr[stack[-1]] >= val:
                mid = stack.pop()
                left_boundary = stack[-1] if stack else -1

                count_left = mid - left_boundary
                count_right = i - mid

                total_sum = (total_sum + extended_arr[mid] * count_left * count_right) % MOD

            stack.append(i)

        return total_sum
```

25.4 Python-Specific Complexities & Implementation Considerations

25.4.1 1. Integer Modulo Behavior

- In Python, integer overflow is not a concern due to arbitrary-precision integers. However, applying `% (10**9 + 7)` in each iteration keeps the numbers small and fits standard performance targets.

25.4.2 2. Space Optimization

- Appending `[0]` to `arr` takes $\mathcal{O}(n)$ time and space because it copies the list. To optimize this, you can iterate over `range(n + 1)` and use a conditional check like `arr[i] if i < n else 0` to retrieve the current value, keeping auxiliary space limited strictly to the stack.

25.5 Complexity Analysis

Metric	Complexity	Description
Time Complexity	$\mathcal{O}(n)$	Each element is pushed onto the stack once and popped at most once.
Space Complexity	$\mathcal{O}(n)$	The stack requires at most n elements in the worst case (e.g. if the array is sorted in ascending order).

25.6 Common Follow-Up Questions & How to Answer

25.6.1 Q1: How does this problem relate to "Sum of Subarray Ranges" (LeetCode 2104)?

- **Answer**: LeetCode 2104 asks for the sum of `max(b) - min(b)` for all subarrays. This is mathematically equivalent to:

$$\sum \max(b) - \sum \min(b)$$

  We can compute the sum of subarray minimums (using this algorithm) and the sum of subarray maximums (using a monotonic stack with the opposite comparison operators), and subtract the two sums to get the result in $\mathcal{O}(n)$ time.

25.6.2 Q2: Can we solve this problem using Dynamic Programming?

- **Answer**: Yes, we can define `dp[i]` as the sum of the minimums of all subarrays ending at index `i`.
- If we know the index of the first smaller element to the left (let it be `j`), then:
  - For subarrays starting after `j`, the minimum element is `arr[i]` (there are `i - j` such subarrays).
  - For subarrays starting at or before `j`, the minimum element is determined by the minimums of subarrays ending at `j`. The sum of these minimums is exactly `dp[j]`.
- This yields the recurrence: `dp[i] = dp[j] + arr[i] * (i - j)`
- We can use a monotonic stack to find `j` for each `i`, resulting in a very clean $\mathcal{O}(n)$ DP + Stack solution.
- **Code Example (C++)**: `cpp int sumSubarrayMins(std::vector<int>& arr) { int n = arr.size(); std::vector<int> dp(n, 0); std::vector<int> stack; long long total = 0, MOD = 1e9 + 7; for (int i = 0; i < n; ++i) { while (!stack.empty() && arr[stack.back()] >= arr[i]) { stack.pop_back(); } int prev_smaller = stack.empty() ? -1 : stack.back(); dp[i] = (prev_smaller == -1 ? 0 : dp[prev_smaller]) + arr[i] * (i - prev_smaller); dp[i] %= MOD; total = (total + dp[i]) % MOD; stack.push_back(i); } return total; }`

25.7 Pro-Tip: How to Impress the Interviewer

- **Demonstrate Single-Pass Sentinel Flushing**: Presenting the single-pass sentinel flushing technique shows high-level fluency with stacks, simplifying standard 3-pass solutions into a single loop.
- **State DP Duality**: Mentioning that the problem can be reformulated as a Dynamic Programming state recurrence optimized by a Monotonic Stack shows that you can bridge the gap between different algorithmic paradigms (DP and stack-based parsing).